

OTH - Regensburg
Fakultät für Elektro- und Informationstechnik

Hardware/Software-Codedesign

Projekt: FPGA - ZynqAudioDSP

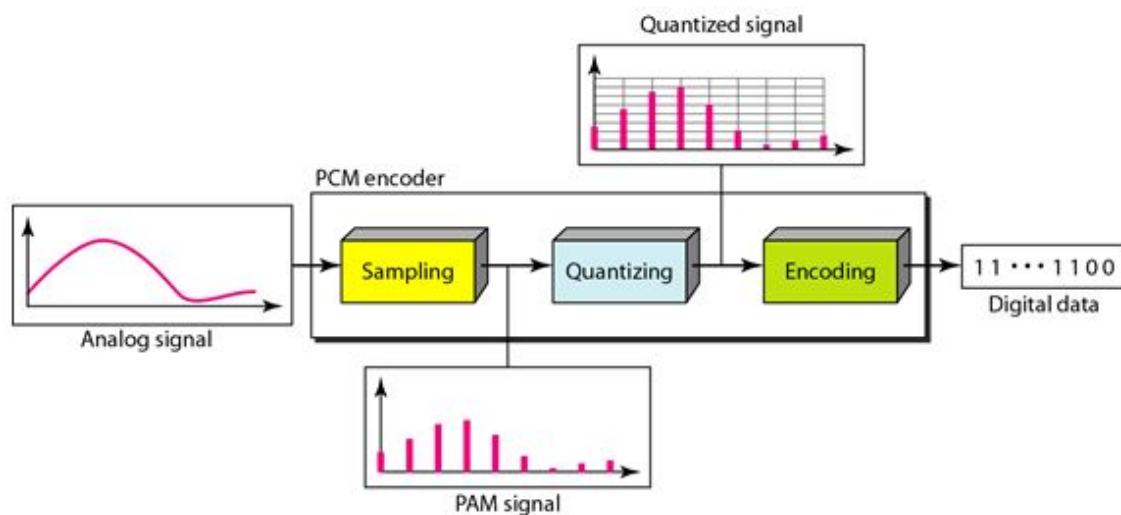


Abbildung 1: DSP - Signaleingangskette

1

Beginn des Projektes: 11.12.2025
Abgabedatum: 23.06.2026
Verfasser des Protokolls: Ralf Höchtel
Betreuer: Prof. Dr.-Ing. Florian Aschauer

¹DSP Input Chain – <https://www.myreadingroom.co.in/notes-and-studymaterial/68-dcn/732-analog-to-digital-conversion-techniques.html>

Inhaltsverzeichnis

1	Vorstellung des Projektes	1
1.1	Motivation	1
1.2	Zielsetzung	1
2	Einführung in die Grundlagen der Audio-Wiedergabe	3
2.1	Arten der Audio-Wiedergabe	3
2.1.1	Puls-Code-Modulation (PCM)	3
2.1.2	Pulsweitenmodulation (PWM)	4
2.2	Soundwiedergabe mittels I ² S-Kommunikation	4
2.2.1	Aufbau der I ² S-Schnittstelle	5
2.2.2	Anbindung eines externen DAC	6
2.2.3	Datenformat und Übertragung eines I ² S-Frames	7
2.3	Systemaufbau und verwendete Hardwareplattform	8
3	Anleitung zur Dokumentation und Projektreproduktion	10
3.1	Projektstruktur im Repository	11
3.2	Reproduktion der Hardwareprojekte mit Vivado	11
3.3	Export der Hardwareplattform für Vitis	14
3.4	Reproduktion der Softwareprojekte mit Vitis	15
3.5	Zusammenfassung des Reproduktionskonzeptes	16
4	Phase 1 – Inbetriebnahme, Verifikation und Vergleich der Audiowiedergabe	17
4.1	Verifikation der Audiokette mit einem Sinus-Testsignal	17
4.1.1	Ziel dieses Arbeitspaketes	17
4.1.2	Umsetzung	17
4.1.3	Funktionsnachweis und Bewertung	18
4.2	Sinus-Keyboad-Demonstration	19
4.2.1	Ziel dieses Arbeitspaketes	19
4.2.2	Umsetzung	19
4.2.3	Funktionsnachweis und Bewertung	20
4.3	PS/PL-basierte Wiedergabe einer WAV-Datei mit Benutzersteuerung	22
4.3.1	Ziel dieses Arbeitspaketes	22
4.3.2	Umsetzung	22
4.3.3	Funktionsnachweis und Bewertung	25
4.4	Ausblick: Untersuchung geeigneter Audioformate	26

5	Phase 2 - Eingangssignalverarbeitung mit Effekten	27
5.1	Hardware-Bypass zur Verifikation des ADC-DAC-Echtzeitpfads	27
5.1.1	Ziel dieses Arbeitspaketes	27
5.1.2	Umsetzung	28
5.1.3	Funktionsnachweis und Bewertung	28
5.2	Entwicklung einer PL-basierten Effektkette	29
5.2.1	Ziel dieses Arbeitspaketes	29
5.2.2	Umsetzung	29
5.2.3	Funktionsnachweis und Bewertung	32
6	Bekannte Probleme und Lessons Learned	34
6.1	Reproduzierbarkeit und Projektstruktur	34
6.2	Schrittweise Inbetriebnahme statt zu komplexer Erstaufbau	34
6.3	Schnittstellentiming und Takterzeugung	35
6.4	Clock-Domain-Crossing bei der PS/PL-Kommunikation	35
6.5	Toolchain- und Umgebungsprobleme	36
6.6	Funktionale Korrektheit und wahrgenommene Audioqualität	37
6.7	Zusammenfassung	37
7	Ausblick und kleines Fazit	38
8	Literatur	40
	Abbildungsverzeichnis	40

1 Vorstellung des Projektes

1.1 Motivation

Die Idee zu diesem Projekt entstand aus dem Wunsch, eigene Effekte für die E-Gitarre und somit einen eigenen Klang zu entwickeln. Dieser Gedanke begleitete mich schon seit einigen Jahren, begonnen hat es mit der Suche nach dem originalen E-Gitarren-Sound aus den 80er Jahren. Aus der Neugier über die Technik dahinter und dem Experimentieren mit diversen Schaltungen, lag die Entscheidung nahe Elektrotechnik zu studieren. Besonders interessiert mich der charakteristische Sound von Röhrenverstärkern und die häufig diskutierte Frage, ob analoge Technik grundsätzlich „besser“ klingt als digitale. Gleichzeitig zeigt die Entwicklung moderner Systeme, dass diese Grenze zunehmend verschwindet: Geräte wie der Kemper Profiler haben bereits vor etwa zehn Jahren demonstriert, wie präzise digitale Signalverarbeitung komplexe Klangcharakteristika nachbilden kann. Aktuelle Gitarrenso- und plattformen wie der Quad Cortex von Neural DSP treiben diese Entwicklung voran und der Unterschied zwischen Digital und Analog scheint mehr und mehr zu verschwimmen. Mit der Wahl der Vertiefungsfächer bietet sich nun die Gelegenheit, ein eigenes Projekt umzusetzen, das diese Interessen verbindet. Gedacht ist die Entwicklung eines Audioprozessors, der sowohl die theoretischen Inhalte aus der Vorlesung zu FPGAs als auch praktische Aspekte der Hard- und Softwareentwicklung zusammenführt. Dabei steht insbesondere die digitale Signalverarbeitung, sowie dessen Entwurf und die Implementierung auf FPGA- und SoC-Plattformen im Mittelpunkt.

1.2 Zielsetzung

Ziel dieses Projektes ist die prototypische Entwicklung eines digitalen Audioprozessors auf Basis des Zynq-SoC (Cora Z7), der eine durchgängige Audiokette vom analogen Eingangssignal, über die digitale Signalverarbeitung, bis hin zur analogen Ausgabe realisiert und sich perspektivisch um Aufnahme- und Effektfunktionen erweitern lässt. Dabei steht sowohl das technische Ergebnis als auch das vertiefte Verständnis der zugrunde liegenden Hard- und Softwarearchitekturen im Vordergrund.

Konkret soll im Rahmen des Projektes ein System entstehen, das eine Audiodatei abrufen kann, über die Zusammenarbeit des ARM-Prozessorkernes (Processing System, PS) mit dem Artix-7-chip (Processing Logic, PL) das Signal verarbeitet und über den I²S-Signalfad auf einen externen Audio-DAC (Pmod I2S2) ausgeben kann. Aufbauend auf dieser Basis wird das System um eine Aufnahmestrecke (Line-In) und einfache digitale Signalverarbeitungsfunktionen zur Implementierung von Effekten erweitert, sodass ein vollständiger nahezu Echtzeit-Audiopfad von der analogen Eingangsstufe bis zur analogen Ausgangsstufe entsteht.

Zur besseren Strukturierung lässt sich das Projekt in mehrere Phasen gliedern:

- **Phase 1 – Inbetriebnahme, Verifikation und Vergleich der Audiowiedergabe:**

Zunächst werden die Entwicklungswerkzeuge eingerichtet und das Cora-Z7-Board in Betrieb genommen. Anschließend wird ein I²S-Sender in der Programmable Logic (PL) implementiert, um den DAC des Pmod-I2S2 anzusteuern. Die Verifikation der Audiokette erfolgt zunächst über die Ausgabe eines im FPGA erzeugten Sinus-Testsignals und anschließend über eine interaktive Tonerzeugung in Form einer Tonleiter. Danach werden PCM-Daten aus internen Audiodateien im Processing System (PS) bereitgestellt, über eine AXI-Schnittstelle an die Programmable Logic übertragen und über den I²S-Wiedergabepfad ausgegeben. Abschließend sollen unterschiedliche Wiedergabekonfigurationen in Bezug auf Abtastrate und Wortbreite miteinander verglichen werden.

- **Phase 2 – Eingangssignalverarbeitung mit Effekten:** In der zweiten Phase wird die Signalkette um einen I²S-Empfänger für den auf dem Pmod I2S2 vorhandenen ADC erweitert. Ziel ist es, ein externes Line-In-Signal zu digitalisieren, im FPGA zu verarbeiten und anschließend wieder über den DAC auszugeben. Zunächst wird ein reiner Durchschleifbetrieb (Bypass) realisiert, um einen stabilen, echtzeitfähigen Pfad von der Eingangs- zur Ausgangsstufe zu gewährleisten. Darauf aufbauend sollen einfache DSP-Funktionen wie Lautstärkeregelung, Mute-Funktion, Gain / Clipping oder ein einfacher Filter (z. B. Tiefpass) implementiert werden, um erste Klangbeeinflussungen zu ermöglichen und Latenz sowie Ressourcenverbrauch zu untersuchen.

- **Phase 3 – Erweiterung um Effekte und Plattformcharakter (Ausblick):** In einer möglichen dritten Phase, die abhängig vom Projektfortschritt eher als Ausblick zu verstehen ist, soll das System zu einer flexiblen Audioeffekt-Plattform weiterentwickelt werden. Dazu gehören die Implementierung weiterer Effekte (z. B. Delay, Verzerrung, Equalizer) sowie eine komfortable Parametrierung über das PS. Langfristig ist zudem eine Portierung auf ein PYNQ-Z2-Board denkbar, um die Steuerung der Effekte über eine höhere Abstraktionsebene (z. B. Python/Jupyter-Oberfläche) zu ermöglichen, oder ein Zybo z7-20, um von größeren Rechenkapazität und mehr vorhandener Peripherie wie on-board audiocodec und Knöpfen / Schieberegler zu profitieren.

Zusammenfassend verfolgt das Projekt das Ziel, ein funktionsfähiges, in Echtzeit arbeitendes digitales Audiosystem aufzubauen und dabei ein tiefes Verständnis für digitale Audiotechnik, I²S-Kommunikation, DSP-Algorithmen sowie die spezifischen Möglichkeiten eines Zynq-SoC mit Kombination aus Prozessor und FPGA-Logik zu erarbeiten.

2 Einführung in die Grundlagen der Audio-Wiedergabe

2.1 Arten der Audio-Wiedergabe

Für die digitale Erzeugung und Ausgabe von Audiosignalen existieren unterschiedliche Verfahren. Im Kontext eingebetteter Systeme sind dabei insbesondere die **Puls-Code-Modulation (PCM)** und die **Pulsweitenmodulation (PWM)** von Bedeutung. Beide Verfahren dienen dazu, ein analog wahrnehmbares Audiosignal mit digitalen Mitteln zu repräsentieren oder zu erzeugen, unterscheiden sich jedoch grundlegend in ihrem Funktionsprinzip.

2.1.1 Puls-Code-Modulation (PCM)

Bei der Puls-Code-Modulation wird ein analoges Signal in festen zeitlichen Abständen abgetastet. Jeder Abtastwert wird anschließend quantisiert und als digitaler Zahlenwert mit einer bestimmten Bitbreite dargestellt. PCM beschreibt damit die Amplitude des Signals zu diskreten Zeitpunkten.

Die Qualität eines PCM-Signals wird im Wesentlichen durch zwei Parameter bestimmt: die **Abtastrate** und die **Bitauflösung**. Die Abtastrate legt fest, wie viele Samples pro Sekunde erfasst oder ausgegeben werden, während die Bitauflösung die Genauigkeit der Amplitudendarstellung bestimmt. Im Audibereich sind beispielsweise Abtastraten von 44,1 kHz und 48 kHz weit verbreitet, während in anspruchsvolleren Anwendungen auch 96 kHz oder 192 kHz verwendet werden. Typische Wortbreiten betragen 16 oder 24 Bit. PCM ist das in digitalen Audiosystemen am häufigsten verwendete Verfahren, da Audiodaten in dieser Form direkt gespeichert, verarbeitet und über digitale Audioschnittstellen wie I²S übertragen werden können.

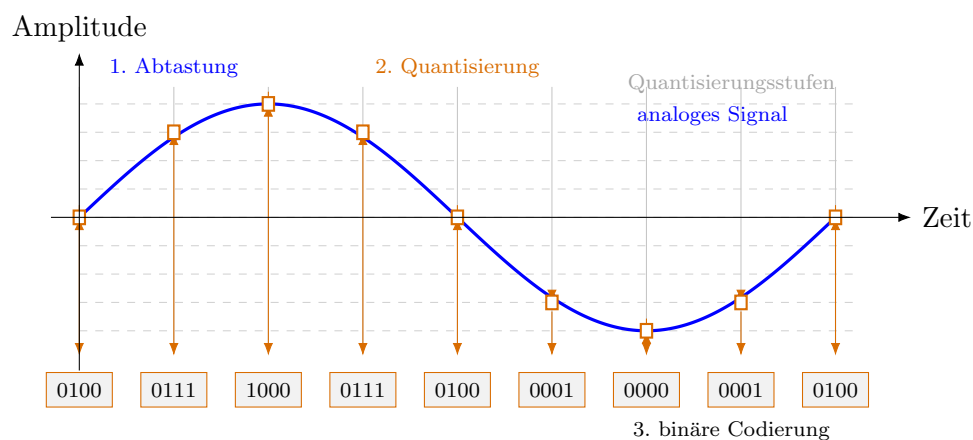


Abbildung 2: Schematische Darstellung der Puls-Code-Modulation (PCM)

2.1.2 Pulsweitenmodulation (PWM)

Im Gegensatz dazu wird bei der Pulsweitenmodulation die Information nicht durch einen mehrbitigen Zahlenwert dargestellt, sondern durch die **Pulsbreite** eines periodischen Rechtecksignals. Die Periodendauer bleibt dabei konstant, während das Verhältnis von Ein-Zeit zu Aus-Zeit (Tastgrad) variiert wird. Der zeitliche Mittelwert dieses Signals entspricht der gewünschten Ausgangsamplitude.

Wird ein PWM-Signal durch einen Tiefpass geglättet, entsteht aus dem Rechtecksignal näherungsweise eine analoge Spannung. Auf diese Weise kann ein Audiosignal auch ohne klassischen Mehrbit-DAC erzeugt werden. PWM eignet sich daher besonders für einfache und kostengünstige Systeme, etwa wenn nur wenige Hardware-Ressourcen zur Verfügung stehen.

Allerdings besitzt PWM gegenüber PCM einige Nachteile. Da zunächst ein hochfrequentes Rechtecksignal erzeugt wird, sind zusätzliche Filtermaßnahmen erforderlich, um das gewünschte Nutzsignal von den Schaltanteilen zu trennen. Zudem ist die erreichbare Audioqualität häufig stärker von Taktfrequenz, Filterung und Störverhalten abhängig, womit der Aufwand für einen klaren Klang stark ansteigt.

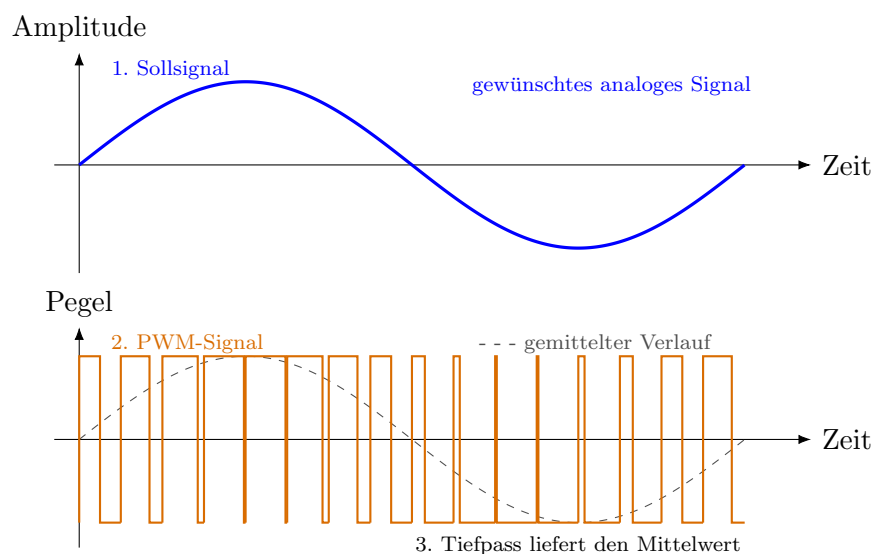


Abbildung 3: Schematische Darstellung einer bipolaren Pulsweitenmodulation (PWM)

2.2 Soundwiedergabe mittels I²S-Kommunikation

Für das vorliegende Projekt ist die Puls-Code-Modulation (PCM) das maßgebliche Verfahren zur Darstellung der Audiodaten. Die im System verarbeiteten Samplewerte werden nicht direkt als analoges Signal ausgegeben, sondern zunächst digital bereitgestellt und anschließend an einen externen Digital-Analog-Wandler übertragen.

Für diese Übertragung wird die *Inter-IC-Sound-Schnittstelle* (I^2S) verwendet. Sie stellt einen weit verbreiteten Standard zur seriellen Übertragung digitaler Audiodaten zwischen integrierten Schaltungen dar. Das I^2S -Protokoll wurde ursprünglich von Philips Semiconductors entwickelt und hat sich als Standard für die serielle Übertragung digitaler Audiodaten zwischen integrierten Schaltungen etabliert. Typische Anwendungsfälle sind die Kommunikation zwischen Mikrocontrollern, FPGAs, Audio-Codecs, ADCs und DACs.

Im vorliegenden Aufbau bildet die I^2S -Schnittstelle die Verbindung zwischen der digitalen Signalverarbeitung in der programmierbaren Logik des Zynq-SoC und dem externen DAC des Pmod-I2S2. Die in der Logik bereitgestellten PCM-Samples werden dabei seriell übertragen und anschließend vom DAC in ein analoges Ausgangssignal umgewandelt. Im Folgenden werden daher zunächst der grundlegende Aufbau der I^2S -Schnittstelle, die beteiligten Signale sowie die zeitliche Struktur der Datenübertragung näher betrachtet.

2.2.1 Aufbau der I^2S -Schnittstelle

Ein I^2S -System besteht typischerweise aus einem *Master* und einem oder mehreren *Slave*-Geräten. Der Master erzeugt die notwendigen Taktsignale, während der Slave die Audiodaten empfängt oder sendet. Die Kommunikation erfolgt üblicherweise über drei zentrale Signale:

- **BCLK oder SCLK (Bit Clock / Serial Clock)**

Dieser Takt bestimmt die Geschwindigkeit der seriellen Datenübertragung. Für jedes übertragene Datenbit wird ein Taktimpuls erzeugt.

- **LRCLK oder WS (Left-Right Clock / Word Select)**

Dieses Signal kennzeichnet, ob gerade ein Sample des linken oder rechten Audiokanals übertragen wird.

- **SD (Serial Data)**

Über diese Leitung werden die seriellen Audiodaten übertragen.

Zusätzlich existiert häufig ein weiterer Takt:

- **MCLK (Master Clock)**

Ein hochfrequenter Referenztakt, der von vielen Audio-DACs zur internen Takterzeugung verwendet wird.

Die Beziehung zwischen diesen Takten hängt von der Abtastrate f_s und der Wortbreite N der übertragenen Samples ab. Für ein Stereo-Signal ergibt sich der Bit-Takt zu

$$BCLK = 2 \cdot N \cdot f_s \quad (1)$$

wobei der Faktor 2 die beiden Audiokanäle eines Stereo-Signals berücksichtigt.

Bei einer beispielhaften Konfiguration mit einer Abtastrate von $f_s = 48\text{ kHz}$ und einer Wortbreite von $N = 24$ Bit ergibt sich damit:

$$BCLK = 2 \cdot 24 \cdot 48\,000 = 2\,304\,000\text{ Hz} = 2,304\text{ MHz} \quad (2)$$

Der Bit-Clock beträgt in diesem Fall also 2,304 MHz.

2.2.2 Anbindung eines externen DAC

In diesem Projekt wird ein externer Audio-DAC über ein I²S-Interface an die programmierbare Logik des Zynq-SoC angebunden. Der FPGA-Teil fungiert dabei als I²S-Master und generiert die notwendigen Taktsignale sowie den seriellen Datenstrom.

Der grundlegende Ablauf der Audiowiedergabe ist dabei:

- Eine Audiodatei (z. B. WAV) wird im Processing System (ARM-Core) bereitgestellt.
- Die enthaltenen PCM-Samples werden über eine AXI-Schnittstelle an die Programmable Logic übertragen.
- Ein in der PL implementierter I²S-Sender serialisiert die Samples und erzeugt die notwendigen Taktsignale.
- Der externe DAC empfängt die Daten über die I²S-Leitungen und wandelt sie in ein analoges Signal um.
- Das analoge Signal kann anschließend über eine analoge Ausgangsstufe wiedergegeben werden.

Für die experimentelle Umsetzung wird ein Pmod-I2S2 verwendet, der sowohl einen Audio-DAC als auch einen Audio-ADC bereitstellt. Das Modul besitzt eine zweireihige 12-Pin-Schnittstelle, über die die Ein- und Ausgänge für die digitale Audioübertragung sowie die Spannungsversorgung herausgeführt sind. Im in diesem Abschnitt betrachteten Wiedergabepfad wird zunächst die DAC-Richtung genutzt, sodass die vom FPGA erzeugten I²S-Signale an den D/A-Wandler des Pmod-I2S2 übertragen werden.

Abbildung 4 zeigt die schematische Anbindung zwischen FPGA und Pmod-I2S2 für die Audioausgabe. Dargestellt sind die wesentlichen Signalleitungen *MCLK*, *SCLK*, *LRCK* und *SDOUT/SDIN* sowie die Versorgungsanschlüsse. Der FPGA erzeugt dabei die Taktsignale und stellt die seriellen PCM-Daten bereit, während das Pmod-I2S2 diese Daten empfängt und in ein analoges Ausgangssignal umwandelt.

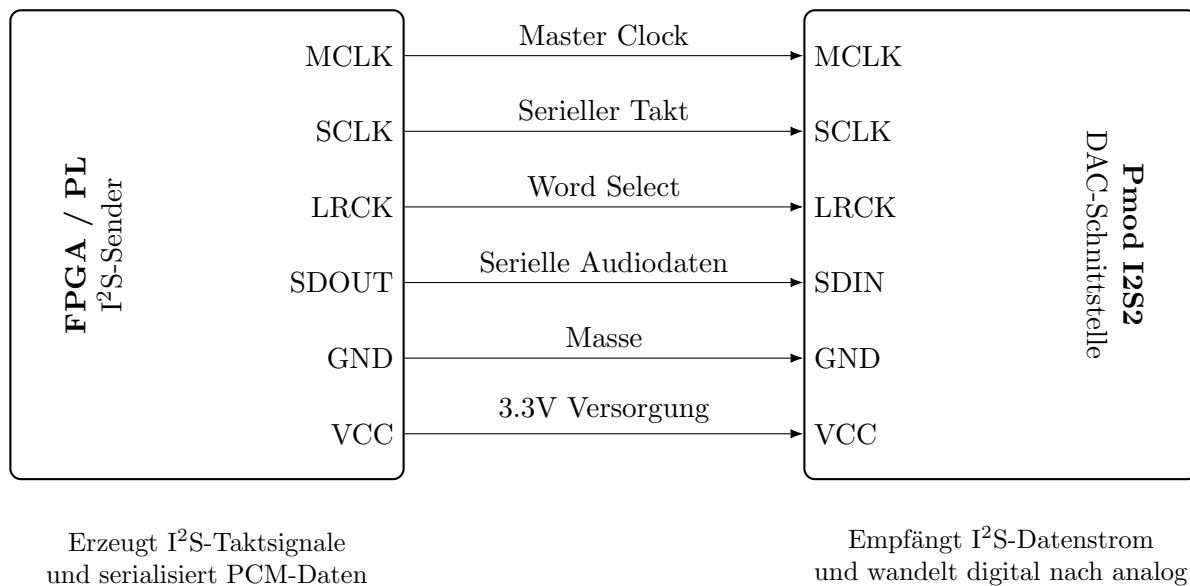


Abbildung 4: Schematische Darstellung der I²S-Verbindung zwischen FPGA-I²S-Sender und externem DAC auf dem Pmod-I2S2.

2.2.3 Datenformat und Übertragung eines I²S-Frames

Nachdem zuvor die grundlegenden Signale der I²S-Schnittstelle eingeführt wurden, wird im Folgenden deren zeitliches Zusammenspiel anhand eines vollständigen Stereo-Frames betrachtet. Dabei sind insbesondere das Datenformat, die Zuordnung zu linkem und rechtem Kanal sowie die zeitliche Lage der seriellen Audiodaten relativ zu den Taktsignalen von Bedeutung.

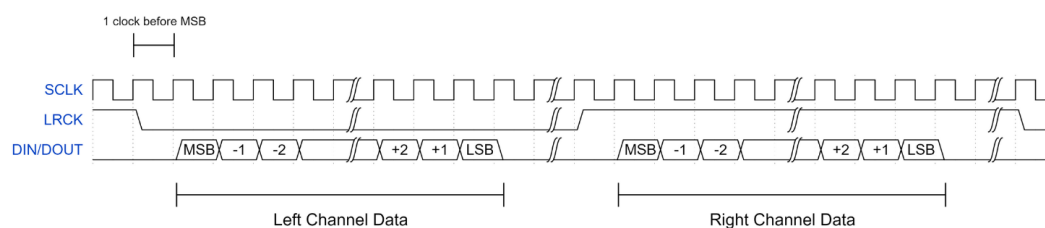


Abbildung 5: Zeitdiagramm der I²S-Datenübertragung eines Stereo-Frames²

Ein vollständiger I²S-Frame besteht aus zwei aufeinanderfolgenden Datenworten, die jeweils einem Audiokanal zugeordnet sind. Die Übertragung erfolgt seriell, synchron zum Bit-Takt **SCLK**. Die Trennung zwischen linkem und rechtem Kanal wird durch das Word-Select-Signal **LRCK** vorgenommen. Eine vollständige Periode dieses Signals entspricht somit genau einem Stereo-Frame.

²Quelle: Pmod I2S2 Reference Manual <https://digilent.com/reference/pmod/pmodi2s2/reference-manual>

Wie in Abbildung 5 dargestellt, werden im gezeigten Fall bei $LRCK = Low$ die Daten des linken Kanals und bei $LRCK = High$ die Daten des rechten Kanals übertragen. Innerhalb jedes Kanalwortes werden die Daten im *MSB-first*-Format gesendet. Das bedeutet, dass zunächst das höchstwertige Bit (*Most Significant Bit, MSB*) und anschließend die weiteren Bits bis zum niedrigstwertigen Bit (*Least Significant Bit, LSB*) übertragen werden.

Ein charakteristisches Merkmal des I²S-Protokolls besteht darin, dass das erste gültige Datenbit eines neuen Wortes nicht unmittelbar mit der Flanke des $LRCK$ -Signals erscheint, sondern erst um einen Bit-Takt verzögert auf der Datenleitung anliegt. Diese zeitliche Verschiebung erleichtert dem Empfänger die eindeutige Erkennung des Beginns eines neuen Datenwortes.

Die eigentlichen Audiodaten werden somit bitweise mit jeder Taktperiode des seriellen Takts übertragen. Nach Abschluss des ersten Kanalwortes wechselt das $LRCK$ -Signal seinen Zustand und das Datenwort des zweiten Kanals folgt unmittelbar. Anschließend beginnt direkt der nächste Frame mit dem folgenden Stereo-Sample.

Die dargestellte zeitliche Struktur bildet die Grundlage für die im weiteren Verlauf des Projekts verwendete Audioausgabe über den I²S-Sender in der programmierbaren Logik.

2.3 Systemaufbau und verwendete Hardwareplattform

Die experimentelle Umsetzung des Audiowiedergabesystems erfolgt auf Basis eines Zynq-SoC-Entwicklungsboards vom Typ *Cora Z7*. Die verwendete Hardwareplattform ist in Abbildung 6 dargestellt. Die in der Abbildung mit **1** markierte Komponente ist das Cora-Z7-Board, das als zentrales Rechen- und Entwicklungssystem dient. Die mit **2** markierte Komponente ist das Audio-Erweiterungsmodul *Pmod-I2S2*, das die externe Audio-Ein- und -Ausgabe bereitstellt.

Das Cora-Z7-Board übernimmt im vorliegenden Aufbau die zentrale Systemfunktion. Es stellt sowohl das Processing System zur Ausführung softwarebasierter Anteile als auch die programmierbare Logik zur Implementierung zeitkritischer Hardwarekomponenten bereit. Die Erzeugung des I²S-Datenstroms erfolgt in der programmierbaren Logik, während softwareseitige Funktionen, wie beispielsweise die Bereitstellung oder Verwaltung von Audiodaten, auf dem Processing System realisiert werden können.

Die Kopplung beider Baugruppen erfolgt über die in den vorherigen Abschnitten beschriebenen I²S-Signale sowie die erforderlichen Versorgungsanschlüsse. Der hier gewählte Aufbau bildet die technische Grundlage für die nachfolgende Verifikation der Audiokette sowie für spätere Erweiterungen um Aufnahmefunktionen und Signalverarbeitungsblöcke. Nachdem damit die verwendete Hardwareplattform und die grundlegende Audioschnittstelle beschrieben wurden, wird im folgenden Kapitel erläutert, wie die praktische Umsetzung

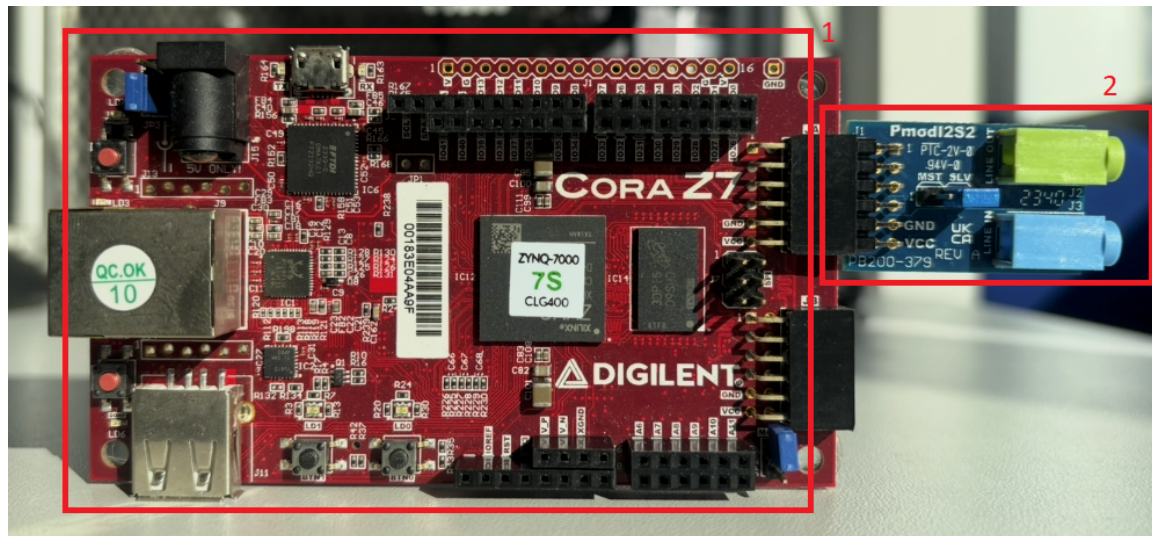


Abbildung 6: Verwendete Hardwareplattform:

- (1) Cora-Z7-Entwicklungsboard mit Zynq-SoC
- (2) Pmod-I2S2 als externe Audio-Peripherie

des Projektes dokumentiert und reproduzierbar strukturiert wurde. Dazu gehören sowohl die Gliederung der einzelnen Entwicklungsschritte als auch die Ablage der Hardware- und Softwarestände im zugehörigen Repository.

3 Anleitung zur Dokumentation und Projektreproduktion

Nachdem in den vorherigen Kapiteln die Grundlagen der digitalen Audiowiedergabe, die PCM-Darstellung von Audiosignalen sowie die Kommunikation über die I²S-Schnittstelle erläutert wurden, beschreibt dieses Kapitel den Aufbau der Projektdokumentation und den Ablauf zur Reproduktion der einzelnen Entwicklungsstände. Ziel ist es, die praktische Umsetzung des Projektes nicht nur inhaltlich zu beschreiben, sondern auch so zu strukturieren, dass die einzelnen Arbeitsschritte anhand der zugehörigen Projektdateien nachvollzogen und bei Bedarf erneut aufgebaut werden können.

Sämtliche im Projekt entwickelten Hardwarebeschreibungen, die zugehörigen Vivado-Skripte sowie die softwareseitigen Komponenten wurden versionskontrolliert in einem öffentlich zugänglichen GitHub-Repository abgelegt. Dadurch ist der Aufbau des Systems reproduzierbar dokumentiert, zur öffentlichen Einsicht verfügbar und als Grundlage für eine weiterführende Entwicklung nutzbar.

Ergänzend dazu wurden Diagramme, Messaufnahmen, Audiodateien, Demonstrationsvideos und weitere projektrelevante Materialien in einem öffentlich zugänglichen Google-Drive-Ordner gesammelt. Diese Unterlagen dienen der transparenten Dokumentation des Entwicklungsverlaufs und ermöglichen einen nachvollziehbaren Vergleich der erzielten Ergebnisse.

GitHub-Repository: <https://github.com/R4lfreichts/fpga-audiodsp>

Google-Drive-Ordner:

<https://drive.google.com/drive/folders/1UXgKQ8-ymGQQzzHMuRiuA8H1rMEDAqNA?usp=sharing>

Ab diesem Punkt wird das Projekt schrittweise dokumentiert. Jeder größere Entwicklungsschritt wird dabei als eigenes Arbeitspaket betrachtet und beschreibt jeweils eine funktionale Erweiterung des Gesamtsystems. Die Ausarbeitung verfolgt dabei bewusst nicht das Ziel, jede einzelne Codezeile detailliert zu erklären. Stattdessen liegt der Fokus auf der Systemarchitektur, der Funktionsweise der entwickelten Module, der Aufgabenteilung zwischen Hardware und Software sowie dem praktischen Nachweis der Funktion. Die konkreten Implementierungsdetails sind ergänzend im Quellcode dokumentiert und können über das Repository nachvollzogen werden.

Die folgenden Abschnitte orientieren sich daher an einer wiederkehrenden Grundstruktur:

- **Ziel des Arbeitspaketes:** Beschreibung, welche Funktionalität im jeweiligen Entwicklungsschritt erreicht werden soll und welche Rolle dieser Schritt im Gesamtsystem einnimmt.

- **Umsetzung:** Erläuterung der gewählten Hardware- und gegebenenfalls Softwarestruktur. Dabei wird beschrieben, welche Module, Schnittstellen oder IP-Blöcke verwendet wurden und wie diese im System zusammenwirken.
- **Funktionsnachweis und Bewertung:** Darstellung der praktischen Testergebnisse sowie eine kurze Bewertung des erreichten Entwicklungsstandes. Hierbei werden insbesondere Funktionalität, Stabilität, Klangqualität und mögliche Einschränkungen betrachtet.

Der Umfang der einzelnen Abschnitte variiert abhängig von der Komplexität des jeweiligen Arbeitspaketes. Ein einfacher Hardwaretest, wie die Ausgabe eines Sinustons, benötigt beispielsweise weniger Erklärungsumfang als ein Entwicklungsschritt, in dem Processing System, DMA, AXI-Stream und I²S-Ausgabe gemeinsam verwendet werden. Die wiederkehrende Struktur soll dennoch sicherstellen, dass die einzelnen Projektstände vergleichbar und nachvollziehbar dokumentiert sind.

3.1 Projektstruktur im Repository

Zur besseren Nachvollziehbarkeit wurde das Projekt in eine klar gegliederte Ordnerstruktur aufgeteilt. Dabei werden Hardwarebeschreibung, Vivado-Skripte, Software, Audiodateien und Dokumentation voneinander getrennt. Die folgende Übersicht zeigt den prinzipiellen Aufbau des Repositorys:

Die Trennung zwischen `hw/rtl` und `hw/scripts` wurde bewusst gewählt. Im Ordner `rtl` befinden sich die eigentlichen VHDL-Quelldateien der jeweiligen Entwicklungsstände. Die dazugehörigen Vivado-Skripte liegen getrennt davon im Ordner `scripts`. Dadurch bleiben Quellcode und Projekterzeugung übersichtlich getrennt, während die Zuordnung über gleich benannte Unterordner erhalten bleibt.

Zusätzlich werden in `constraints` die Board-spezifischen Constraint-Dateien abgelegt. Diese beschreiben unter anderem die Zuordnung der logischen Ports zu den physikalischen Pins des Entwicklungsboards. Der Ordner `bitstreams` enthält ausgewählte lauffähige Bitstream-Dateien, die zum schnellen Testen einzelner Projektstände verwendet werden können. Die Softwareanteile für das Processing System befinden sich im Ordner `sw`.

3.2 Reproduktion der Hardwareprojekte mit Vivado

Um die einzelnen Hardwarestände möglichst unabhängig von lokal erzeugten Vivado-Projektdateien reproduzierbar zu machen, werden die Projekte über Tcl-Skripte aufgebaut. Dadurch muss nicht das vollständige Vivado-Projekt mit allen automatisch generierten Dateien versioniert werden. Stattdessen reicht es aus, die VHDL-Quellen, die Constraint-Dateien und die zugehörigen Skripte im Repository abzulegen.

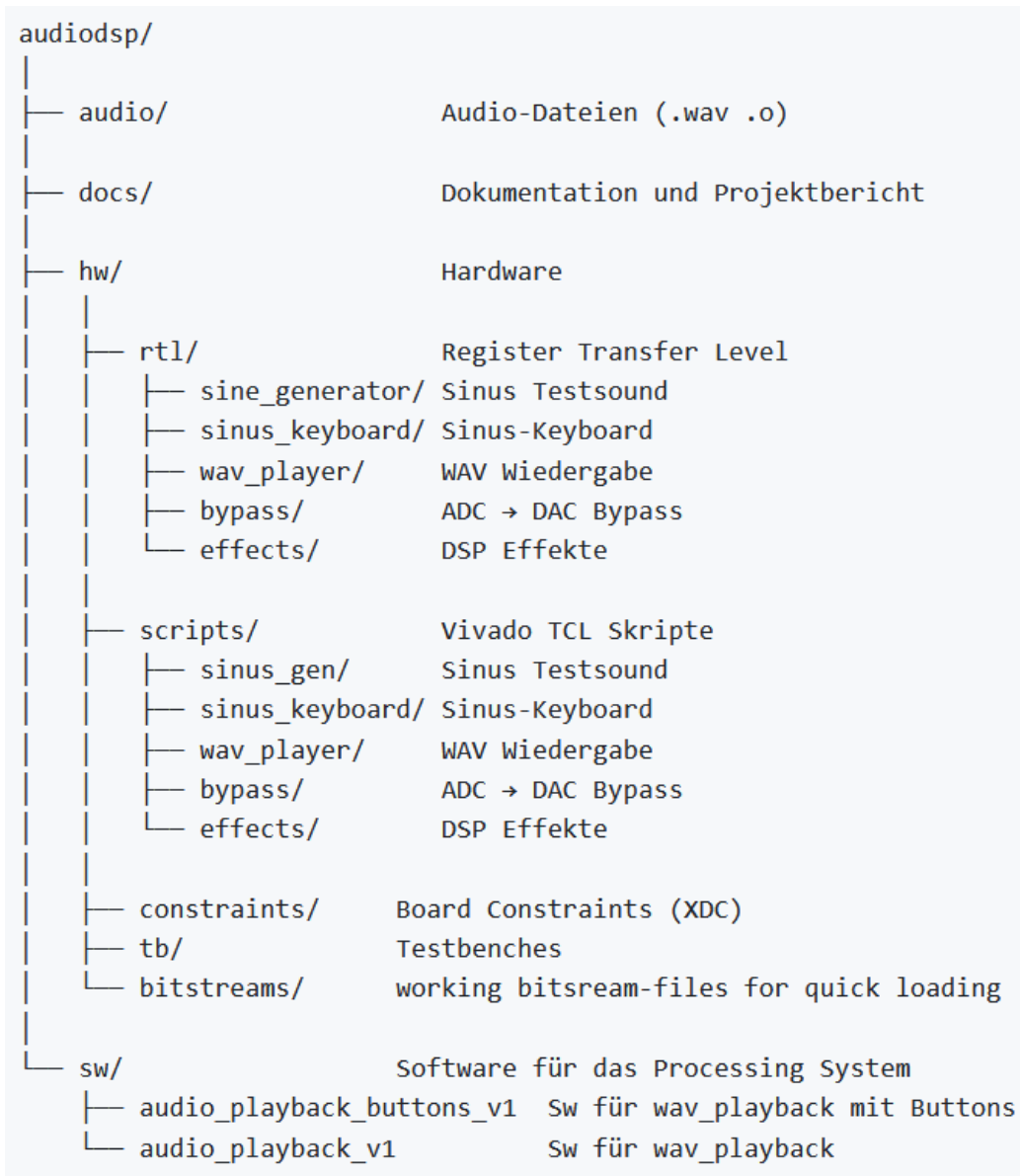
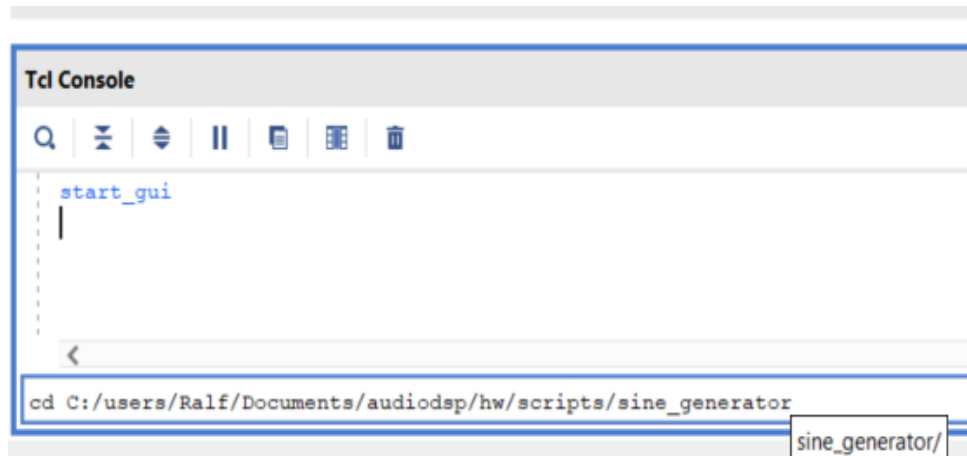


Abbildung 7: Struktur des Repositorys

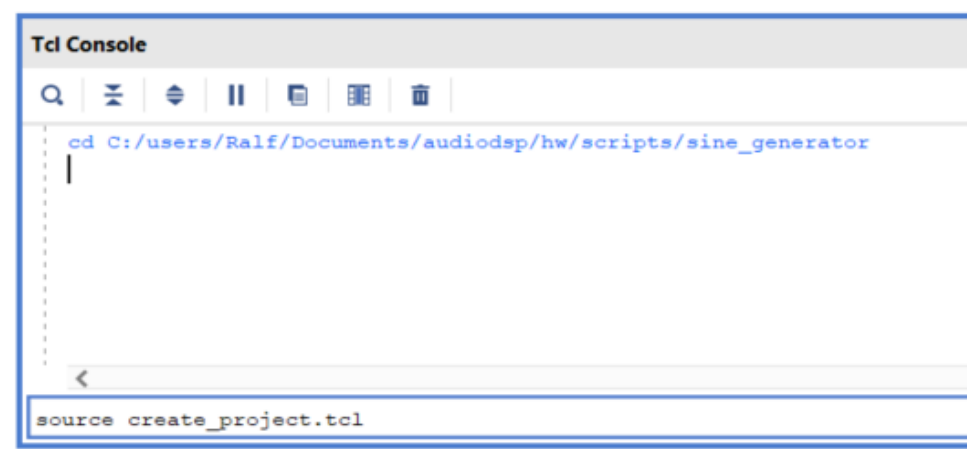
Der grundsätzliche Ablauf zur Reproduktion eines Hardwarestandes ist bei den einzelnen Projektabschnitten ähnlich. Zunächst wird Vivado gestartet. Anschließend wird in der Vivado Tcl Console in den jeweiligen Skriptordner gewechselt. Dort kann das Projekt mit dem zugehörigen Erzeugungsskript neu aufgebaut werden.

Beispielhaft kann der Ablauf wie folgt aussehen:

```
cd C:/Pfad/zum/Repository/audiodsp/hw/scripts/wav_player
source create_project.tcl
```



(a) change directory



(b) create project

Abbildung 8: Vivado Projekt erstellen

Das Skript `create_project.tcl` erzeugt das Vivado-Projekt, bindet die benötigten VHDL-Dateien ein, lädt die Constraint-Dateien und legt die passende Top-Level-Entity fest. Falls ein Blockdesign oder zusätzliche IP-Blöcke benötigt werden, werden diese ebenfalls über die zugehörigen Skripte erzeugt und in das Projekt eingebunden. Auf diese Weise entsteht aus den versionierten Projektdaten ein vollständiges Vivado-Projekt, ohne dass die grafische Benutzeroberfläche manuell durchlaufen werden muss.

Nach erfolgreicher Projekterzeugung können wie gewohnt Synthese, Implementierung und Bitstream-Erzeugung durchgeführt werden. Der allgemeine Ablauf lautet damit:

- Vivado starten.
- In der Tcl Console in den passenden Skriptordner wechseln.
- Das Projektskript mit `source create_project.tcl` ausführen.
- Synthese und Implementierung starten.

- Bitstream erzeugen.
- Bei Bedarf die Hardwareplattform als `.xsa`-Datei exportieren.

Diese Vorgehensweise hat den Vorteil, dass die Projektstände über verschiedene Systeme hinweg deutlich einfacher reproduzierbar sind. Außerdem wird vermieden, dass große automatisch generierte Vivado-Projektordner versioniert werden müssen. Entscheidend ist lediglich, dass das Repository vollständig geklont wurde und die relative Ordnerstruktur unverändert bleibt.

3.3 Export der Hardwareplattform für Vitis

Sobald ein Projektstand neben der Hardwarebeschreibung auch eine Softwarekomponente auf dem ARM-Prozessor des Zynq-SoC verwendet, muss die erzeugte Hardwareplattform aus Vivado exportiert werden. Dies geschieht über eine sogenannte `.xsa`-Datei. Diese Datei enthält die Beschreibung der Hardwareplattform und kann zusätzlich den erzeugten Bitstream enthalten.

Die `.xsa`-Datei bildet die Schnittstelle zwischen Vivado und Vitis. Während Vivado für die Erstellung der Hardware zuständig ist, wird in Vitis auf Basis dieser exportierten Hardwareplattform die Software für das Processing System erstellt. Die Software benötigt diese Informationen, um beispielsweise die vorhandenen AXI-Peripherieblöcke, Adressbereiche, Interrupts und Schnittstellen korrekt ansprechen zu können.

Der grundsätzliche Ablauf lautet daher:

- Hardwareprojekt in Vivado erzeugen.
- Synthese, Implementierung und Bitstream-Erzeugung durchführen.
- Hardwareplattform inklusive Bitstream als `.xsa`-Datei exportieren.
- In Vitis eine passende Plattform auf Basis dieser `.xsa`-Datei erstellen.
- Eine Softwareapplikation anlegen und den zugehörigen C-Code einbinden.
- Software kompilieren und auf dem Board ausführen.

Damit ergibt sich für alle Projektstände mit Softwareanteil ein wiederkehrender Workflow:

Hardware in Vivado erstellen → `.xsa`-Datei exportieren → Software in Vitis erstellen → System auf dem Board testen

3.4 Reproduktion der Softwareprojekte mit Vitis

Die Softwareanteile des Projektes befinden sich im Ordner `sw`. Dort sind die jeweiligen C-Quelldateien, Build-Dateien und zusätzliche benötigte Dateien abgelegt. Dazu können beispielsweise `main.c`, `CMakeLists.txt`, ein Linkerskript oder eingebettete Audiodaten gehören.

Im Gegensatz zur Hardwareseite wurde für die Software bewusst keine vollständig automatisierte Erzeugung über Skripte verwendet. Während die Vivado-Projekte mit Tcl-Skripten zuverlässig reproduzierbar aufgebaut werden konnten, traten bei der softwareseitigen Projekterzeugung zwischen unterschiedlichen Vitis-Versionen eher Kompatibilitätsprobleme auf. Daher wurde für die Software ein robusterer und praktisch besser nachvollziehbarer Weg gewählt.

Die Plattform und die Applikation müssen mit Vitis neu angelegt werden. Anschließend werden die im Repository bereitgestellten Quelldateien in die Applikation eingebunden. Falls sich die Hardware geändert hat, muss außerdem die zugehörige `.xsa`-Datei aktualisiert und die Plattform in Vitis entsprechend neu erzeugt oder angepasst werden.

Für die Reproduktion eines Softwarestandes ergibt sich damit folgender Ablauf:

- In Vivado die passende Hardwareversion erzeugen und als `.xsa`-Datei exportieren.
- In Vitis eine neue Plattform auf Basis dieser `.xsa`-Datei erstellen.
- Eine neue Bare-Metal-Applikation für den ARM-Prozessor anlegen.
- Die im Ordner `sw` bereitgestellten Quelldateien in das Projekt übernehmen.
- Gegebenenfalls zusätzliche Dateien wie Audiodaten oder Linkerskripte einbinden.
- Software kompilieren.
- Bitstream laden und die Applikation auf dem Board starten.

Diese Vorgehensweise ist zwar manueller als die Erzeugung der Vivado-Hardwareprojekte, hat sich im Projektverlauf jedoch als zuverlässiger erwiesen. Gleichzeitig bleibt die Reproduktion nachvollziehbar, da alle wesentlichen Quell- und Konfigurationsdateien im Repository abgelegt sind.

Für den erfolgreichen Build- und Ausführungsprozess ist es erforderlich, dass die Dateien `main.c` und `CMakeLists.txt` aus dem `sw`-Ordner sowie die Objektdaten `endoftime_audio_wav.o` aus dem `audio`-Ordner im `src`-Verzeichnis der neu angelegten Vitis-Plattform bzw. Applikation hinterlegt sind (siehe Abbildung 9). Anschließend kann das Projekt kompiliert und ausgeführt werden.

Name	Änderungsdatum	Typ
.cache	12.04.2026 23:17	Dateiordner
.compile_commands	11.04.2026 17:44	Dateiordner
.clangd	11.04.2026 17:44	CLANGD-Datei
app.yaml	11.04.2026 17:44	Yaml-Quelldatei
audio_app.bif	12.04.2026 23:39	BIF-Datei
BOOT.bin	20.06.2026 23:34	BIN-Datei
CMakeLists.txt	23.06.2026 12:53	Textdokument
compile_commands.json	23.06.2026 12:53	JSON-Quelldatei
Empty_applicationExample.cmake	11.04.2026 17:44	CMake-Quelldatei
endoftime_audio_wav.o	08.04.2026 00:33	O-Datei
lscript.ld	11.04.2026 17:44	LD-Datei
main.c	12.04.2026 23:31	C-Quelldatei
README.txt	13.10.2023 17:03	Textdokument

Abbildung 9: Erforderliche Dateistruktur im `src`-Verzeichnis der Vitis-Applikation.

3.5 Zusammenfassung des Reproduktionskonzeptes

Das Projekt ist so aufgebaut, dass die einzelnen Entwicklungsstände möglichst klar voneinander getrennt und bei Bedarf erneut aufgebaut werden können. Die Hardwareseite wird über versionierte VHDL-Dateien, Constraint-Dateien und Tcl-Skripte reproduziert. Die Softwareseite wird über die im Repository abgelegten C-Quellen und zugehörigen Konfigurationsdateien bereitgestellt und in Vitis auf Basis der exportierten Hardwareplattform neu eingebunden.

Damit ergibt sich eine klare Trennung zwischen den einzelnen Ebenen des Projektes:

- **Vivado** erzeugt die Hardwareplattform und den Bitstream.
- Die **.xsa-Datei** übergibt die Hardwarebeschreibung an Vitis.
- **Vitis** erzeugt die Software für das Processing System.
- Das **Repository** dokumentiert die jeweiligen Projektstände über Quellcode, Skripte und ergänzende Dateien.

Diese Struktur unterstützt sowohl die eigene Weiterentwicklung des Projektes als auch die spätere Nachvollziehbarkeit der Ergebnisse. Die Ausarbeitung selbst beschreibt dabei die übergeordneten Konzepte, Systemzusammenhänge und Testergebnisse, während das Repository die konkrete technische Umsetzung der einzelnen Projektstände bereitstellt.

4 Phase 1 – Inbetriebnahme, Verifikation und Vergleich der Audiowiedergabe

In der ersten Projektphase wurde die grundlegende Audiowiedergabe des Systems schrittweise in Betrieb genommen und verifiziert. Ziel dieser Phase war es zunächst, den digitalen Wiedergabepfad über die programmierbare Logik und den externen DAC des Pmod-I2S2 funktionsfähig aufzubauen. Darauf aufbauend wurde die Audiokette anschließend um einfache Demonstrations- und Wiedergabefunktionen erweitert, um die Grundlage für spätere Hardware-/Software- und DSP-Erweiterungen zu schaffen.

4.1 Verifikation der Audiokette mit einem Sinus-Testsignal

4.1.1 Ziel dieses Arbeitspaketes

Der erste praktische Entwicklungsschritt bestand darin, die grundlegende Audiokette des Systems in Betrieb zu nehmen. Ziel war es, einen einfachen und kontrollierbaren Testton vollständig in der programmierbaren Logik zu erzeugen und über den externen DAC des Pmod-I2S2 auszugeben. Dadurch sollte überprüft werden, ob die wesentlichen Bestandteile des Wiedergabepfades korrekt zusammenarbeiten.

Im Mittelpunkt stand dabei noch keine komplexe Audiodatenverarbeitung, sondern der reine Funktionsnachweis der digitalen Audioausgabe. Insbesondere sollten die Takterzeugung, die I²S-konforme Serialisierung der Audiodaten, die Signalführung zwischen FPGA und Pmod-I2S2 sowie die analoge Ausgabe über den DAC überprüft werden. Ein Sinussignal eignet sich hierfür besonders gut, da es einfach zu erzeugen, eindeutig hörbar und messtechnisch leicht auswertbar ist.

Dieser Schritt bildet damit die Grundlage für alle weiteren Projektabschnitte. Erst wenn ein stabiler Wiedergabepfad vom FPGA bis zum analogen Ausgang nachgewiesen ist, können darauf aufbauend komplexere Signalquellen wie eine WAV-Wiedergabe oder später Echtzeit-Effekte sinnvoll integriert werden.

4.1.2 Umsetzung

Die Umsetzung des Sinus-Testsignals erfolgt vollständig in der programmierbaren Logik. Der Aufbau besteht im Wesentlichen aus den beiden VHDL-Dateien `i2s_playback.vhd` und `i2s_transceiver.vhd`. Die Datei `i2s_playback.vhd` bildet dabei die oberste Entwurfsebene dieses Projektstandes. Sie bindet den Clock-Wizard ein, erzeugt das digitale Testsignal und übergibt die resultierenden PCM-Samplewerte an den I²S-Sender.

Für die Erzeugung des Testsignals wird ein digitaler Phasenakkumulator verwendet. Bei jedem neuen Stereo-Sample wird ein festes Phaseninkrement addiert. Aus dem aktuellen

Phasenwert wird anschließend ein Tabellenindex für eine Sinus-Lookup-Tabelle gebildet. Zur Reduzierung des Speicherbedarfs ist dabei nur ein Viertel der Sinusperiode hinterlegt; die vollständige Sinusform wird über die Quadranten des Phasenwertes rekonstruiert. Der berechnete 24-Bit-Samplewert wird identisch für den linken und rechten Audiokanal verwendet. Die serielle Ausgabe übernimmt das Modul `i2s_transceiver.vhd`. Dieses erzeugt aus dem Master-Clock die benötigten I²S-Signale `sclk` und `ws`, übernimmt die linken und rechten Samplewerte und schiebt diese MSB-first über die Datenleitung `sd_tx` zum DAC des Pmod-I2S2. Zusätzlich stellt das Modul ein `sample_tick`-Signal bereit, mit dem die Signalerzeugung in `i2s_playback.vhd` jeweils genau einmal pro Stereo-Frame aktualisiert wird.

Damit ist die Audiokette klar getrennt aufgebaut: `i2s_playback.vhd` erzeugt das digitale PCM-Testsignal, während `i2s_transceiver.vhd` die zeitkritische I²S-Übertragung übernimmt. Diese Trennung wird in den folgenden Arbeitspaketen beibehalten, sodass spätere Erweiterungen hauptsächlich die Signalquelle betreffen, während der I²S-Ausgabepfad unverändert weiterverwendet werden kann.

4.1.3 Funktionsnachweis und Bewertung

Der Funktionsnachweis erfolgte zunächst durch die hörbare Ausgabe des erzeugten Sinustons. Damit konnte grundsätzlich bestätigt werden, dass der implementierte I²S-Pfad, die Verbindung zum Pmod-I2S2 sowie die Ausgabe über den DAC funktionieren. Ein stabil hörbarer Ton zeigt bereits, dass die wesentlichen Taktsignale erzeugt werden und die seriellen Audiodaten vom DAC korrekt interpretiert werden.

Zur genaueren Überprüfung wurde das ausgegebene Audiosignal zusätzlich messtechnisch ausgewertet. Nach Anpassung des Frequenzparameters ergab die Messung eine dominante Frequenz von etwa 440,2 Hz. Damit liegt das erzeugte Signal sehr nahe am angestrebten Kammerton A mit 440 Hz. Die verbleibende Abweichung ist für diesen Funktionstest vernachlässigbar und bestätigt, dass die Takterzeugung sowie das zyklische Auslesen der Sinuswerte hinreichend genau arbeiten. Insgesamt konnte mit diesem Arbeitspaket die grundlegende Audiokette erfolgreich verifiziert werden.

Hinweis: Ein Video zur Demonstration dieses Projektstandes ist unter dem Namen „Sinusgenerator Aufzeichnung“ im Google Drive hinterlegt.

Mit diesem Arbeitspaket wurde somit ein stabiler und wiederverwendbarer I²S-Wiedergabepfad geschaffen. Für die folgenden Arbeitspakete muss die DAC-Ansteuerung daher nicht erneut grundlegend betrachtet werden. Stattdessen kann jeweils beschrieben werden, welche neue Signalquelle oder Steuerlogik vor diesen bestehenden Ausgabepfad geschaltet wird.



Abbildung 10: Frequenzmessung des erzeugten Testsignals

4.2 Sinus-Keyboard-Demonstration

4.2.1 Ziel dieses Arbeitspaketes

Nach der erfolgreichen Verifikation des I²S-Wiedergabepfades mit einem festen Sinus-Testsignal sollte die bisher statische Tonerzeugung um eine einfache Benutzerinteraktion erweitert werden. Ziel war es, nicht mehr nur einen einzelnen Sinuston auszugeben, sondern über zwei Taster zwischen mehreren Tonhöhen umschalten zu können.

Dazu wurde eine einfache Sinus-Keyboard-Demonstration umgesetzt. Über die Taster des Entwicklungsboards kann eine C-Dur-Tonleiter durchlaufen werden. Dieses Arbeitspaket dient damit als kleiner Funktionsnachweis dafür, dass der zuvor aufgebaute Audiopfad nicht nur ein festes Testsignal ausgeben kann, sondern auch auf eine Benutzereingabe reagieren und unterschiedliche digitale Tonsignale erzeugen kann.

4.2.2 Umsetzung

Die Sinus-Keyboard-Demonstration baut direkt auf dem in Abschnitt 4.1 verifizierten Wiedergabepfad auf. Die grundlegende I²S-Ausgabe über `i2s_transceiver.vhd` wurde beibehalten. Dadurch musste die DAC-Ansteuerung nicht erneut verändert werden. Angepasst wurde hauptsächlich die Datei `i2s_playback.vhd`, in der die Signalerzeugung und die Tastersteuerung zusammengeführt werden.

Die Tonerzeugung basiert weiterhin auf einem digitalen Phasenakkumulator mit Sinus-Lookup-Tabelle. Im Unterschied zum vorherigen Arbeitspaket wird jedoch nicht mehr nur ein festes Phaseninkrement für einen einzelnen Testton verwendet. Stattdessen sind in `i2s_playback.vhd` mehrere Phaseninkremente für die Töne der C-Dur-Tonleiter hinterlegt. Abhängig vom aktuell ausgewählten Tonindex wird das passende Inkrement verwendet und daraus der jeweilige Sinuston erzeugt.

Die Ausgabe der Audiodaten erfolgt unverändert über `i2s_transceiver.vhd`. Das Modul übernimmt weiterhin die Erzeugung der I²S-Signale `sclk`, `ws` und `sd_tx` und überträgt die 24-Bit-Samplewerte an den DAC des Pmod-I2S2. Damit bleibt die Schnittstellenlogik vom eigentlichen Tonerzeugungsblock getrennt.

Für die Tasterauswertung wird zusätzlich das Modul `button_onepulse.vhd` verwendet. Dieses synchronisiert und entprellt die Eingangssignale der Taster und erzeugt pro erkanntem Tastendruck genau einen kurzen Impuls. Diese Impulse werden in `i2s_playback.vhd` genutzt, um den aktuellen Tonindex entweder um eine Stufe zu erhöhen oder zu verringern. Dadurch kann die Tonleiter schrittweise vorwärts und rückwärts durchlaufen werden.

Der Aufbau bleibt damit modular: `button_onepulse.vhd` bereitet die Tastereingaben auf, `i2s_playback.vhd` wählt den passenden Ton und erzeugt die PCM-Samplewerte, während `i2s_transceiver.vhd` die unveränderte I²S-Ausgabe an den externen DAC übernimmt.

4.2.3 Funktionsnachweis und Bewertung

Der Funktionsnachweis erfolgte durch die praktische Wiedergabe der einzelnen Töne über den Audioausgang des Pmod-I2S2. Durch Betätigung der Taster konnten die Tonhöhen hörbar verändert und die Töne der C-Dur-Tonleiter nacheinander ausgegeben werden. Damit wurde gezeigt, dass die erweiterte Signalerzeugung, die Tasterauswertung und der bestehende I²S-Ausgabepfad korrekt zusammenarbeiten.

Zusätzlich wurde die Funktion in einem Demonstrationsvideo festgehalten, das im zugehörigen Google-Drive-Ordner abgelegt ist. Dieses Video dient als ergänzender Nachweis der praktischen Funktionsfähigkeit.

Während der Tests zeigte sich außerdem, dass die wahrgenommene Klangqualität nicht nur von der digitalen Implementierung abhängt, sondern auch von der Versorgung des Entwicklungsboards beeinflusst wird. Bei Versorgung über einen Laptop war teilweise ein leicht kratziger Klang hörbar. Bei Versorgung über ein externes Netzteil beziehungsweise Ladekabel wirkte die Ausgabe dagegen sauberer. Dies deutet darauf hin, dass Störeinflüsse oder Spannungsschwankungen der Versorgung einen hörbaren Einfluss auf den analogen Ausgangspfad haben können.

Insgesamt konnte mit diesem Arbeitspaket gezeigt werden, dass sich der zuvor verifizierte Audiopfad einfach um eine benutzergesteuerte Tonerzeugung erweitern lässt. Die Umset-

zung ist bewusst einfach gehalten, zeigt aber anschaulich das Potenzial der entwickelten Audioplattform. Würden die synthetisch erzeugten Sinustöne zukünftig durch gespeicherte Samples realer Instrumente ersetzt, könnte auf derselben Grundlage bereits ein sehr einfaches digitales Keyboard entstehen.

Im nächsten Arbeitspaket wird die bisher rein hardwareseitige Signalerzeugung durch eine softwaregestützte Audiodatenbereitstellung erweitert. Ziel ist es dann, eine Audiodatei im Processing System bereitzustellen und deren PCM-Daten an die programmierbare Logik zu übertragen.

Hinweis: Ein Video zur Demonstration dieses Projektstandes ist unter dem Namen „Sinus-Keyboard Aufzeichnung“ im Google Drive hinterlegt.

4.3 PS/PL-basierte Wiedergabe einer WAV-Datei mit Benutzersteuerung

4.3.1 Ziel dieses Arbeitspaketes

Nach der rein hardwareseitigen Tonerzeugung bestand der nächste Entwicklungsschritt darin, eine echte Audiodatei wiederzugeben. Ziel war es, eine im Processing System bereitgestellte WAV-Datei einzulesen, deren PCM-Daten softwareseitig aufzubereiten und anschließend an die programmierbare Logik zu übertragen. Die Ausgabe sollte weiterhin über den bereits verifizierten I²S-Wiedergabepfad und den externen DAC des Pmod-I2S2 erfolgen.

Damit wurde erstmals die Zusammenarbeit zwischen Processing System und Programmable Logic praktisch genutzt. Das Processing System übernimmt dabei die datenorientierten Aufgaben wie WAV-Auswertung, Speicherverwaltung, Benutzersteuerung und DMA-Transfer. Die programmierbare Logik bleibt dagegen für den kontinuierlichen und zeitkritischen Audiopfad zuständig.

Zusätzlich sollte die Wiedergabe über die Taster des Entwicklungsboards gesteuert werden können. Vorgesehen waren eine Play/Pause-Funktion sowie das Zurücksetzen der Wiedergabe auf den Anfang der Audiodatei.

4.3.2 Umsetzung

Ein wesentlicher Vorbereitungsschritt der softwareseitigen Audiowiedergabe besteht zunächst darin, die verwendete WAV-Datei nicht zur Laufzeit aus einem Dateisystem einzulesen, sondern bereits während des Buildprozesses in die Anwendung einzubinden. Hierzu wird die Audiodatei in eine Objektdatei (.o) überführt und anschließend beim Linken gemeinsam mit den übrigen Programmdateien in das resultierende ELF eingebunden. Dadurch liegt der Inhalt der Audiodatei nach dem Programmstart bereits als zusammenhängender Speicherbereich im Adressraum der Anwendung vor.

Der Verweis auf diese eingebettete Objektdatei erfolgt über die Vitis-/CMake-Konfiguration. In der Software können anschließend über die vom Linker bereitgestellten Symbole Zeiger auf den Anfang und das Ende der eingebetteten Audiodatei verwendet werden. In `main.c` wird daraus zunächst ein Zeiger auf den Beginn der WAV-Datei sowie über die Differenz der beiden Adressen die Dateilänge bestimmt. Auf diese Weise kann die Datei im weiteren Verlauf direkt als Bytefeld im Speicher verarbeitet werden, ohne dass ein Dateisystem oder ein externer Massenspeicherzugriff erforderlich ist.

Im Zentrum des Systems steht das *Processing System* 7. Dieses stellt den ARM-Prozessor, den Zugriff auf den DDR-Speicher sowie die AXI-Schnittstellen zur programmierbaren Logik bereit. Die Software läuft als Bare-Metal-Anwendung auf dem ARM-Prozessor. Die

Audiodatei wird dabei nicht von einem Dateisystem gelesen, sondern als eingebettetes Objekt in die Anwendung eingebunden. Der entsprechende Verweis auf die Objektdaten erfolgt über die Vitis-/CMake-Konfiguration.

Zu Beginn prüft die Software in `main.c` den WAV-Header. Dabei werden unter anderem Audioformat, Kanalanzahl, Abtastrate, Wortbreite sowie Position und Größe des Datenbereichs ausgewertet. Im umgesetzten Projektstand wird eine Stereo-PCM-Datei mit 48 kHz und 24 Bit verwendet. Ein Stereo-Frame besteht damit aus zwei 24-Bit-Samples, also insgesamt 6 Byte Nutzdaten pro Abtastzeitpunkt.

Der Hardwareaufbau wurde in Vivado als Blockdesign umgesetzt. Abbildung 11 zeigt die grundlegende Aufgabenteilung zwischen Processing System und Programmable Logic.

Im Zentrum des Systems steht das *Processing System 7*. Dieses stellt den ARM-Prozessor, den Zugriff auf den DDR-Speicher sowie die AXI-Schnittstellen zur programmierbaren Logik bereit. Die Software läuft als Bare-Metal-Anwendung auf dem ARM-Prozessor. Die Audiodatei wird dabei nicht von einem Dateisystem gelesen, sondern als eingebettetes Objekt in die Anwendung eingebunden. Der entsprechende Verweis auf die Objektdaten erfolgt über die Vitis-/CMake-Konfiguration.

Zu Beginn prüft die Software in `main.c` den WAV-Header. Dabei werden unter anderem Audioformat, Kanalanzahl, Abtastrate, Wortbreite sowie Position und Größe des Datenbereichs ausgewertet. Im umgesetzten Projektstand wird eine Stereo-PCM-Datei mit 48 kHz und 24 Bit verwendet. Ein Stereo-Frame besteht damit aus zwei 24-Bit-Samples, also insgesamt 6 Byte Nutzdaten pro Abtastzeitpunkt.

Für die Übertragung der Audiodaten in die PL wird ein *AXI Direct Memory Access* verwendet. Der DMA arbeitet hier im MM2S-Betrieb (*Memory Mapped to Stream*). Er liest Daten aus dem Speicherbereich des Processing Systems und gibt sie als AXI4-Stream in Richtung der programmierbaren Logik aus. Der Prozessor muss dadurch nicht jedes einzelne Sample selbst übertragen, sondern startet nur blockweise DMA-Transfers.

Die Software bereitet die Audiodaten in Blöcken fester Größe auf. Im vorliegenden Stand werden jeweils 256 Stereo-Frames in einen Übertragungspuffer geschrieben. Aus den beiden 24-Bit-Samples eines Stereo-Frames wird dabei ein 64-Bit-Datenwort für den AXI-Stream gebildet. Die unteren 32 Bit enthalten das linke Sample, die oberen 32 Bit das rechte Sample. Vor dem Start des DMA-Transfers wird der Daten-Cache für diesen Pufferbereich geleert, damit der DMA die aktuellen Daten aus dem Speicher liest.

Der erzeugte AXI-Stream wird zunächst an eine *AXI-Stream FIFO* übergeben. Diese FIFO ist ein zentraler Bestandteil des Designs. Sie puffert die blockweise vom DMA gelieferten Daten und entkoppelt diese von der kontinuierlichen Audioausgabe. Der DMA arbeitet datenblockorientiert und wird von der Software gestartet. Der I²S-Pfad benötigt dagegen

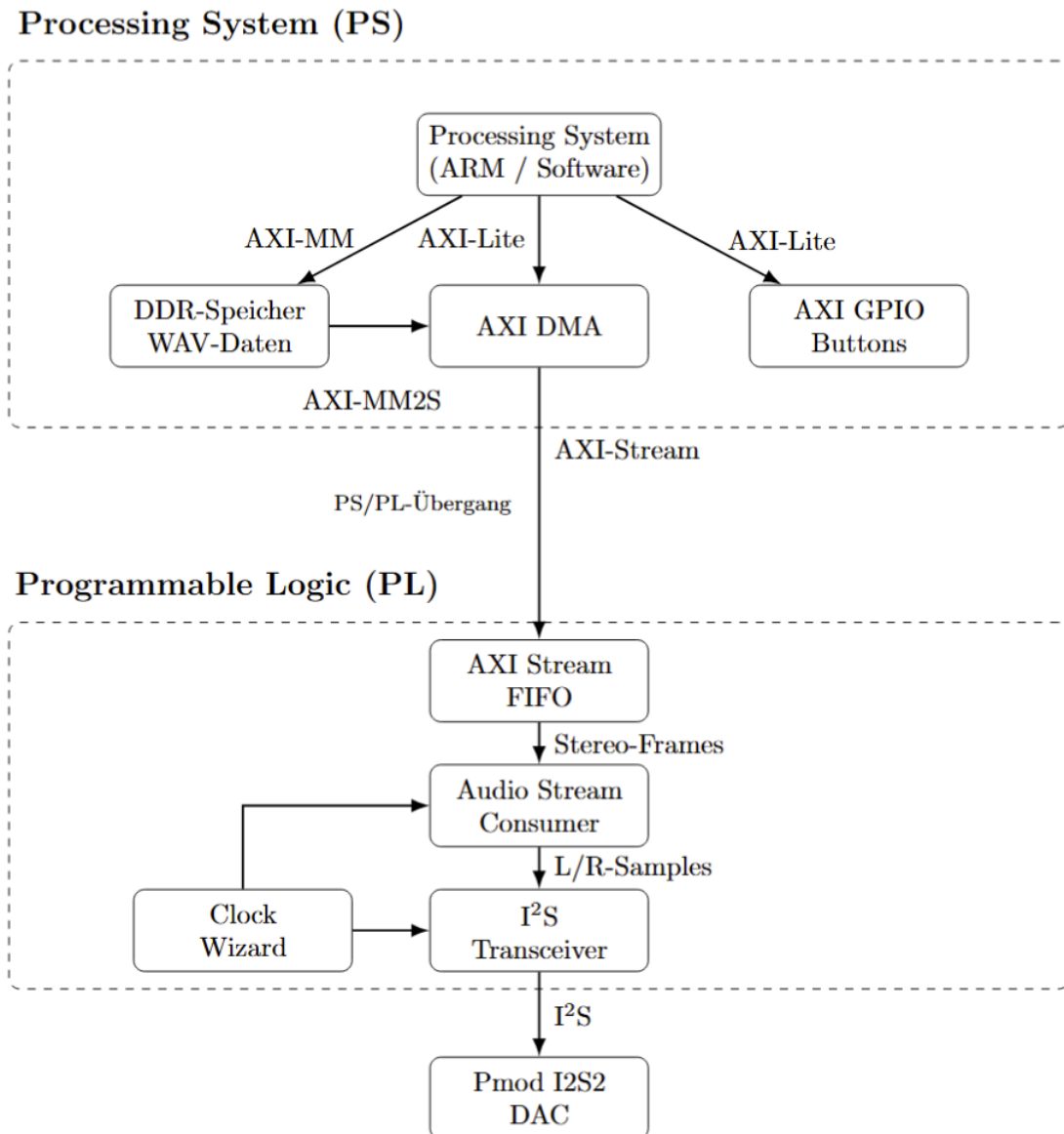


Abbildung 11: Vereinfachtes Blockdiagramm der WAV-basierten Audiowiedergabe mit Aufgabentrennung zwischen PS und PL.

in festen zeitlichen Abständen neue Samples. Die FIFO gleicht diese unterschiedlichen Arbeitsweisen aus.

Zusätzlich dient die FIFO als Übergang zwischen den Taktdomänen. Die Eingangsseite der FIFO ist mit dem AXI-Systemtakt des Processing Systems verbunden. Die Ausgangsseite arbeitet dagegen mit dem Audiotakt, der im Modul `i2s_audio_sink.vhd` erzeugt wird. Dadurch kann der Stream aus dem PS-Bereich kontrolliert in den eigentlichen Audiopfad der PL überführt werden.

Der selbst entwickelte Audioblock besteht im Wesentlichen aus `i2s_audio_sink.vhd`, `audio_stream_consumer.vhd` und `i2s_transceiver.vhd`. Das Modul `i2s_audio_sink.vhd` bildet die Schnittstelle zwischen Blockdesign und I²S-Ausgabe. Es erzeugt über den Clock-Wizard den Audiotakt, gibt diesen auch an die Ausgangsseite der FIFO weiter und bindet den Stream-Consumer sowie den I²S-Transceiver ein.

Das Modul `audio_stream_consumer.vhd` übernimmt die 64-Bit-Stereo-Frames aus dem AXI-Stream. Über das AXI-Stream-Handshake-Prinzip wird dabei nur dann ein neues Datenwort übernommen, wenn gültige Daten anliegen und der interne Puffer frei ist. Anschließend werden aus dem Datenwort das linke und rechte 24-Bit-Sample entnommen und dem I²S-Pfad bereitgestellt. Falls zum benötigten Zeitpunkt kein gültiges neues Frame vorhanden ist, gibt das Modul Stille aus und setzt ein `underrun`-Signal.

Die eigentliche serielle Ausgabe erfolgt weiterhin über `i2s_transceiver.vhd`. Dieses Modul erzeugt die Signale `sclk` und `ws` und überträgt die linken und rechten Samplewerte über `sd_tx` an den DAC des Pmod-I2S2. Damit wird der bereits aus den vorherigen Arbeitspaketen bekannte I²S-Ausgabepfad weiterverwendet.

Für die Benutzersteuerung wird ein AXI-GPIO-Block eingesetzt. Die beiden Taster des Entwicklungsboards werden von der Software zyklisch ausgelesen. Ein Taster schaltet zwischen Wiedergabe und Pause um, der andere setzt die aktuelle Wiedergabeposition auf den Anfang der Audiodatei zurück. Bei pausierter Wiedergabe werden keine neuen DMA-Transfers gestartet.

4.3.3 Funktionsnachweis und Bewertung

Der Funktionsnachweis erfolgte durch die praktische Wiedergabe der eingebetteten WAV-Datei über den Audioausgang des Pmod-I2S2. Die Audiodaten konnten erfolgreich im Processing System vorbereitet, per AXI-DMA in die programmierbare Logik übertragen und über den I²S-Ausgabepfad wiedergegeben werden.

Auch die Benutzersteuerung konnte verifiziert werden. Über die Taster ließ sich die Wiedergabe starten, pausieren und auf den Anfang zurücksetzen. Zusätzlich wurde die Funktion in einem Demonstrationsvideo festgehalten, das im zugehörigen Google-Drive-Ordner

abgelegt ist.

Die Audioausgabe erwies sich im Test als stabil und qualitativ überzeugend. Besonders wichtig ist an diesem Arbeitspaket jedoch die erfolgreiche Datenkette zwischen Software und Hardware. Der DMA übernimmt den effizienten Transport größerer Datenmengen, die FIFO puffert und entkoppelt den AXI-Stream vom festen Audiotiming, und die PL sorgt weiterhin für die zeitkritische Ausgabe an den DAC. Damit wurde eine zentrale Grundlage für die folgenden Projektphasen geschaffen. Das System kann nun nicht mehr nur intern erzeugte Testsignale ausgeben, sondern echte Audiodaten aus dem Processing System in die programmierbare Logik übertragen. Auf dieser Basis lassen sich später weitere Softwaresteuerungen, andere Audioformate oder zusätzliche Signalverarbeitungsblöcke integrieren.

Hinweis: Ein Video zur Demonstration dieses Projektstandes ist unter dem Namen „WAV-Playback Aufzeichnung“ im Google Drive hinterlegt, sowie zwei Bilder mit dem UART-Output.

4.4 Ausblick: Untersuchung geeigneter Audioformate

Im weiteren Verlauf des Projektes soll untersucht werden, welche Kombination aus Abtastrate und Bitauflösung für das vorliegende System einen sinnvollen Kompromiss zwischen Audioqualität, Datenaufkommen und Implementierungsaufwand darstellt. Geplant ist dabei ein Vergleich verschiedener Konfigurationen, beispielsweise 48 kHz, 96 kHz und perspektivisch auch 192 kHz. Die Abtastrate von 192 kHz ist dabei besonders interessant, da sie der maximal unterstützten Abtastrate des verwendeten Pmod-I2S2 entspricht und somit die obere Grenze der eingesetzten Audiohardware darstellt.

Dazu sollen Audiosignale mit verschiedenen Parametern erzeugt beziehungsweise aufbereitet und anschließend über die bestehende Audiokette wiedergegeben werden. Die Bewertung soll sowohl anhand der wahrgenommenen Klangqualität als auch anhand der technischen Anforderungen erfolgen. Besonders relevant sind dabei die entstehende Datenrate, die Belastung des Processing Systems, die Anforderungen an DMA-Transfers und FIFO-Pufferung sowie mögliche Auswirkungen auf die spätere Echtzeitverarbeitung.

Aus zeitlichen Gründen konnte diese Untersuchung im Rahmen des aktuellen Projektstandes noch nicht umgesetzt werden. Sie bleibt jedoch ein sinnvoller nächster Schritt, da die Wahl des Audioformats direkten Einfluss auf die spätere Erweiterung um Aufnahmefunktionen und DSP-Effekte hat.

5 Phase 2 - Eingangssignalverarbeitung mit Effekten

Nach der in Phase 1 erfolgreich verifizierten Audiowiedergabe über den DAC-Pfad des Pmod-I2S2 wurde das System im nächsten Entwicklungsschritt um einen Echtzeitpfad für externe Eingangssignale erweitert. Ziel dieser Projektphase ist es, ein analoges Audiosignal über den ADC des Pmod-I2S2 einzulesen, in der programmierbaren Logik des Zynq-SoC weiterzuverarbeiten und anschließend wieder über den DAC als analoges Ausgangssignal auszugeben. Damit wird die zuvor ausschließlich auf Wiedergabe ausgerichtete Audiokette zu einem bidirektionalen Signalpfad erweitert, der als Grundlage für nachfolgende DSP-Funktionen dient.

Im Unterschied zur zuvor realisierten WAV-Wiedergabe mit Beteiligung von Processing System und Programmable Logic wurde dieser Entwicklungsschritt wieder vollständig in der programmierbaren Logik umgesetzt. Dadurch konnte ein direkter, taktsynchroner und möglichst latenzarmer Signalpfad aufgebaut werden, der ohne DMA, AXI-Streaming oder softwareseitige Laufzeitsteuerung auskommt. Für die hier betrachtete Echtzeitverarbeitung ist diese reine PL-Umsetzung zunächst sinnvoll, da die Audiodaten unmittelbar im Hardwarepfad verarbeitet werden.

Für spätere Erweiterungen kann eine erneute Einbindung des Processing Systems dennoch vorteilhaft sein. Insbesondere Funktionen mit größerem Speicherbedarf, beispielsweise Looping, Delay-Effekte oder eine komfortablere Parametrierung, könnten von der Kombination aus PL-basierter Echtzeitverarbeitung und PS-seitiger Speicher- bzw. Steuerlogik profitieren.

5.1 Hardware-Bypass zur Verifikation des ADC-DAC-Echtzeitpfads

5.1.1 Ziel dieses Arbeitspaketes

Nach der erfolgreichen Umsetzung der Audiowiedergabe über den DAC-Pfad bestand der nächste Schritt darin, auch die Eingangsrichtung des Pmod-I2S2 in Betrieb zu nehmen. Ziel dieses Arbeitspaketes war es, ein externes Audiosignal über den ADC des Pmod-I2S2 einzulesen, in der programmierbaren Logik entgegenzunehmen und anschließend ohne klangliche Veränderung wieder über den DAC auszugeben.

Damit sollte der vollständige Echtzeitpfad vom analogen Eingang über die I²S-Schnittstelle und die FPGA-Logik bis zurück zum analogen Ausgang verifiziert werden. Im Mittelpunkt stand dabei noch keine eigentliche Signalverarbeitung, sondern der Nachweis, dass Empfang, interne Übergabe und erneute Ausgabe der Audiodaten stabil funktionieren.

Der Bypass bildet damit die Grundlage für das nachfolgende Arbeitspaket `audio_fx`. Erst wenn der reine Durchschleifbetrieb zuverlässig arbeitet, können später gezielt DSP-Blöcke in diesen Signalpfad eingefügt werden.

5.1.2 Umsetzung

Die Umsetzung des Bypass erfolgt vollständig in der programmierbaren Logik. Die zentrale Top-Level-Datei dieses Projektstandes ist `i2s_bypass.vhd`. Sie bindet den Clock-Wizard ein, instanziert den I²S-Transceiver und verbindet die empfangenen Audiodaten direkt wieder mit der Ausgabeseite.

Der Clock-Wizard erzeugt aus dem Boardtakt den benötigten Master-Clock für die I²S-Kommunikation. Dieser wird sowohl für den ADC- als auch für den DAC-Teil des Pmod-I2S2 verwendet. Ebenso werden die Signale `sclk` und `ws` an beide Seiten des Moduls ausgegeben. Das FPGA arbeitet damit weiterhin als I²S-Master und gibt die Taktsignale für Ein- und Ausgabe vor. Die eigentliche serielle Audioübertragung übernimmt erneut das Modul `i2s_transceiver.vhd`. Im Unterschied zu den vorherigen Wiedergabeversuchen wird hier nicht nur `sd_tx` für die Ausgabe genutzt, sondern zusätzlich `sd_rx` für den Empfang der seriellen Audiodaten vom ADC. Das Modul erzeugt aus dem Master-Clock die Signale `sclk` und `ws`, liest die eingehenden Daten kanalweise ein und stellt die empfangenen 24-Bit-Samples als `l_data_rx` und `r_data_rx` bereit.

Die Weitergabe im Bypass selbst ist bewusst minimal gehalten. In `i2s_bypass.vhd` werden bei jedem `sample_tick` die zuletzt empfangenen linken und rechten Samplewerte direkt auf die Sendeseite übernommen. Die Signale `l_data_tx` und `r_data_tx` erhalten also unmittelbar die Werte von `l_data_rx` und `r_data_rx`. Der `sample_tick` wird vom Transceiver einmal pro vollständigem Stereo-Frame erzeugt und markiert damit den Zeitpunkt, an dem neue gültige Samples zur Verfügung stehen.

Durch diese Struktur entsteht ein direkter ADC-DAC-Pfad mit nur sehr geringer Verarbeitung zwischen Eingang und Ausgang. Gleichzeitig ist die Stelle, an der die Samples im Bypass übernommen werden, bereits der spätere Einfügapunkt für DSP-Funktionen. Anstelle der direkten Zuweisung können in einem nächsten Schritt beispielsweise Verstärkung, Mute-Funktion, Filter oder andere Effekte eingefügt werden.

5.1.3 Funktionsnachweis und Bewertung

Der Funktionsnachweis erfolgte durch das Einspeisen eines externen Audiosignals am Line-In-Eingang des Pmod-I2S2 und die anschließende Wiedergabe über den Audioausgang. Das Eingangssignal konnte erfolgreich digitalisiert, in der FPGA-Logik entgegengenommen und wieder über den DAC ausgegeben werden. Damit wurde bestätigt, dass die Eingangsrichtung des Pmod-I2S2 sowie der vollständige ADC-DAC-Pfad grundsätzlich funktionieren.

Besonders relevant war hierbei, dass der Signalpfad ohne Beteiligung des Processing Systems arbeitet. Es werden keine DMA-Transfers, keine AXI-Streams und keine softwareseitigen Laufzeitentscheidungen benötigt. Die Audiodaten laufen stattdessen kontinuierlich und

taktsynchron durch die programmierbare Logik. Dadurch eignet sich dieser Aufbau gut als Basis für latenzarme Echtzeitverarbeitung. Die Audioausgabe entsprach im Bypass-Betrieb dem eingespeisten Signal und zeigte damit, dass die Kanalübernahme und die erneute I²S-Ausgabe korrekt zusammenarbeiten. Der Bypass stellt somit einen wichtigen Zwischenschritt dar: Er verifiziert nicht nur den Eingangspfad, sondern definiert auch die spätere Struktur für einfache DSP-Erweiterungen im Projektstand `audio_fx`.

Während der praktischen Tests fiel auf, dass der Ausgangspegel bei Nutzung von einfachen passiven In-Ear-Kopfhörern am Pmod-Ausgang wenige dB leiser wahrgenommen wurde, als am Kopfhörerausgang des Laptops. Eine mögliche Ursache ist, dass das Pmod-I2S2 als Line-In-/Line-Out-Schnittstelle ausgelegt ist und der Ausgangspegel daher nicht speziell auf passive Kopfhörer angepasst ist, wie es bei Laptopausgängen normalerweise der Fall ist.

Hinweis: Ein Video zur Demonstration dieses Projektstandes ist unter dem Namen „Bypass Aufzeichnung“ im Google Drive hinterlegt.

5.2 Entwicklung einer PL-basierten Effektkette

5.2.1 Ziel dieses Arbeitspaketes

Nachdem im vorherigen Arbeitspaket der reine ADC-DAC-Bypass erfolgreich verifiziert wurde, bestand der nächste Schritt darin, diesen Echtzeitpfad um erste Signalverarbeitungsfunktionen zu erweitern. Ziel war es, das eingehende Audiosignal nicht mehr nur unverändert weiterzuleiten, sondern in der programmierbaren Logik gezielt klanglich zu beeinflussen.

Hierzu wurde eine einfache, modular aufgebaute Effektkette entwickelt, die direkt in den bestehenden Echtzeitpfad eingefügt wird. Als erste Effektmodule wurden eine Verstärkungsstufe sowie eine Clipping-Stufe umgesetzt. Dadurch sollte gezeigt werden, dass der zuvor aufgebaute Hardware-Bypass als Grundlage für latenzarme digitale Audioeffekte verwendet werden kann.

Zusätzlich sollte die Effektkette zur Laufzeit beeinflusst werden können. Über die Taster des Entwicklungsboards wird daher die Stärke der Verstärkungsstufe verändert. Da die Verstärkung vor der Clipping-Stufe liegt, beeinflusst sie gleichzeitig auch die Stärke der nachfolgenden Verzerrung. Auf diese Weise lässt sich mit einer einfachen Bedienung bereits eine hörbare Veränderung des Gesamtsignals erzielen.

5.2.2 Umsetzung

Die Effektkette wurde vollständig in der programmierbaren Logik umgesetzt. Die zentrale Top-Level-Datei dieses Projektstandes ist `audio_fx_top.vhd`. In ihr werden die Takterzeu-

gung, der I²S-Empfang und die I²S-Ausgabe, die Tasterauswertung sowie die einzelnen Effektmodule zusammengeführt.

Der grundlegende Signalfluss des Systems lautet:

$$\text{ADC} \rightarrow \text{i2s_transceiver.vhd} \rightarrow \text{fx_gain.vhd} \rightarrow \text{fx_clipping.vhd} \rightarrow \text{i2s_transceiver.vhd} \rightarrow \text{DAC}$$

Das Modul `i2s_transceiver.vhd` übernimmt dabei wie bereits in den vorherigen Projektständen den Empfang und die Ausgabe der Audiodaten. Über die serielle Leitung `sd_rx` werden die vom ADC kommenden I²S-Daten eingelesen und als linkes und rechtes 24-Bit-Sample bereitgestellt. Nach der Verarbeitung in der Effektkette werden die resultierenden Samples über `sd_tx` wieder seriell an den DAC des Pmod-I2S2 ausgegeben. Das vom Transceiver erzeugte `sample_tick`-Signal markiert jeweils den Zeitpunkt, zu dem ein vollständiges neues Stereo-Sample vorliegt und die Effektkette aktualisiert wird.

Die erste Stufe der Effektkette ist das Modul `fx_gain.vhd`. Dieses verändert die Amplitude des Eingangssignals stufenweise. Die Verstärkung wird dabei nicht über eine Multiplikation mit einem frei wählbaren Faktor realisiert, sondern über eine binäre Verschiebung des Samplewertes. Eine Linksverschiebung des digitalen Wertes entspricht, solange keine Sättigung eintritt, einer Multiplikation mit 2 pro verschobenem Bit, da rechts jeweils eine „0“ ergänzt wird. Eine Rechtsverschiebung wirkt entsprechend als Abschwächung. Dadurch lässt sich die Verstärkungsstufe mit sehr geringem Hardwareaufwand umsetzen. Der zugehörige Parameter beschreibt somit allgemein, wie stark das Signal verstärkt oder abgeschwächt wird. Ein kleiner positiver Wert führt zu einer leichten Verstärkung, ein größerer positiver Wert entsprechend zu einer stärkeren Anhebung. Negative Werte wirken als Abschwächung. Abbildung 12 zeigt schematisch, wie sich unterschiedliche Parametereinstellungen auf ein sinusförmiges Eingangssignal auswirken.

Da eine Verstärkung durch Linksverschiebung zu Überläufen führen kann, ist in `fx_gain.vhd` zusätzlich eine Sättigungslogik enthalten. Überschreitet das berechnete Signal den darstellbaren Bereich des 24-Bit-Audioformats, wird es auf den maximal beziehungsweise minimal zulässigen Wert begrenzt. Dadurch werden numerische Überläufe vermieden.

Nach der Verstärkungsstufe folgt das Modul `fx_clipping.vhd`. Dieses Modul begrenzt das Signal auf einen vorgegebenen Amplitudenbereich. Der zugehörige Parameter legt fest, ab welchem Pegel die Begrenzung einsetzt. Liegt das Signal innerhalb dieses Bereichs, bleibt es unverändert. Überschreitet es den positiven oder negativen Grenzwert, wird es auf genau diesen Grenzwert gesetzt. Dadurch entsteht ein sogenanntes hartes Clipping, bei dem die Signalspitzen abrupt abgeschnitten werden.

Die Wirkung dieses Effekts hängt direkt vom gewählten Grenzwert ab. Ein kleinerer Grenzwert führt dazu, dass das Signal früher begrenzt wird, wodurch die Verzerrung

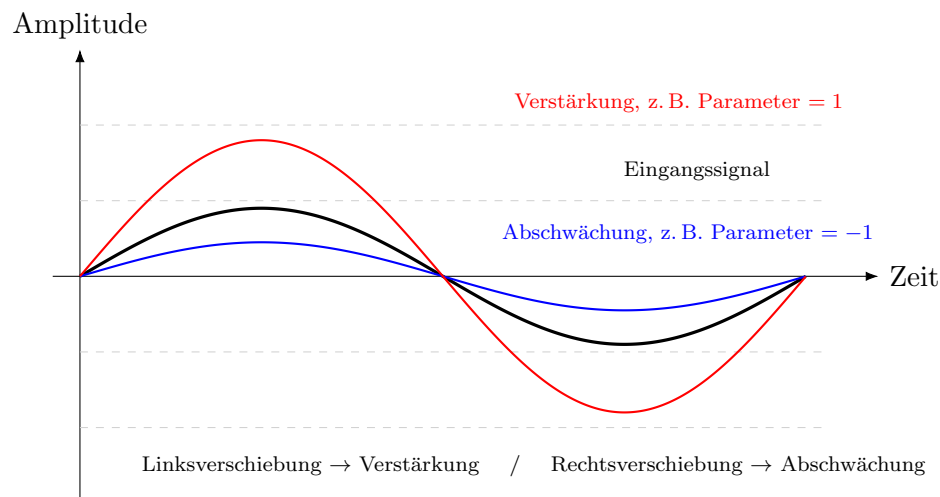


Abbildung 12: Schematische Wirkung der Verstärkungsstufe auf ein sinusförmiges Eingangssignal.

stärker ausfällt. Ein größerer Grenzwert lässt mehr Signalanteile unverändert passieren und bewirkt entsprechend eine schwächere Verzerrung. Abbildung 13 zeigt diesen Effekt exemplarisch an einem sinusförmigen Signal.

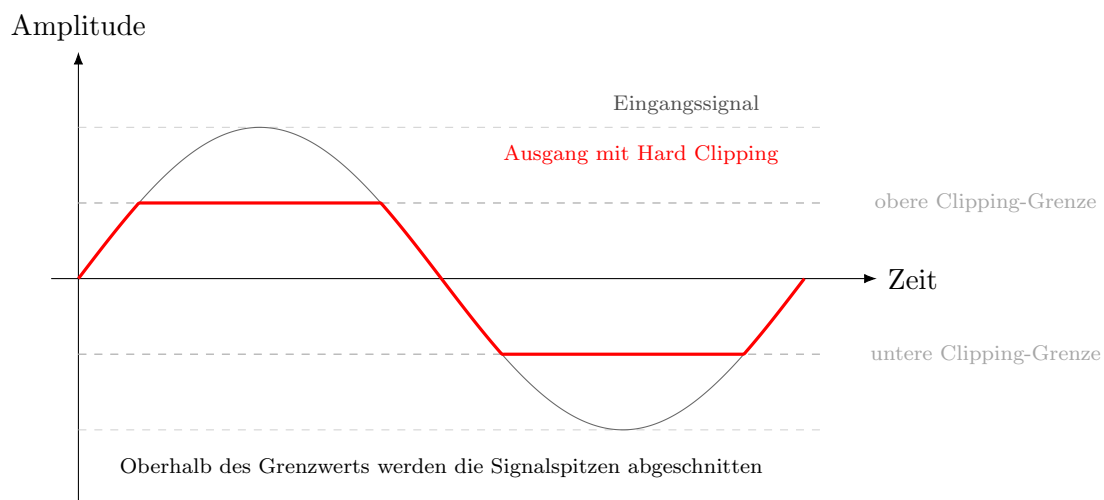


Abbildung 13: Schematische Wirkung einer Hard-Clipping-Stufe auf ein sinusförmiges Eingangssignal.

Zusätzlich wurde mit `fx_softclipping.vhd` eine alternative, weichere Form der Signalbegrenzung vorbereitet. Im Unterschied zum harten Clipping werden die Signalspitzen dort nicht abrupt abgeschnitten, sondern im Bereich nahe der Aussteuerungsgrenze zunehmend abgeflacht. Dadurch entsteht ein sanfterer Übergang in die Sättigung. Die prinzipielle Wirkung ist in Abbildung 14 schematisch dargestellt. Im hier beschriebenen Projektstand ist jedoch das Modul `fx_clipping.vhd` als aktive Stufe in die Effektkette eingebunden. Die Steuerung der Verstärkung erfolgt über zwei Taster des Entwicklungsboards. Hierzu wird erneut das Modul `button_onepulse.vhd` verwendet, das die Tastereingänge synchro-

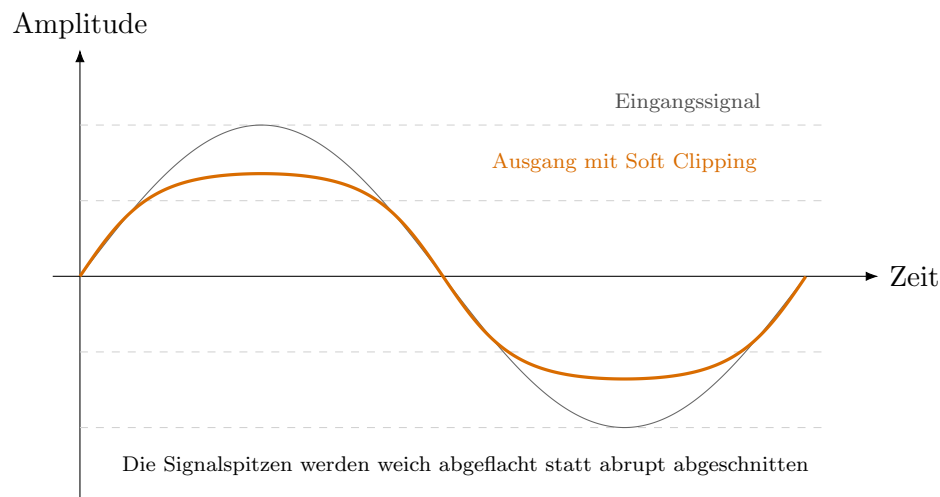


Abbildung 14: Schematische Wirkung einer Soft-Clipping-Stufe auf ein sinusförmiges Eingangssignal.

nisiert, entprellt und pro Tastendruck genau einen kurzen Impuls erzeugt. Diese Impulse verändern in `audio_fx_top.vhd` das Verstärkungsregister schrittweise nach oben oder unten. Da die Clipping-Stufe nach der Verstärkung liegt, beeinflusst diese Verstellung nicht nur die Lautstärke des Signals, sondern gleichzeitig auch, wie stark das Signal in die Begrenzung hineingefahren wird.

Insgesamt ergibt sich damit eine klar strukturierte und modular aufgebaute Effektkette: `i2s_transceiver.vhd` übernimmt den Audioempfang und die Audioausgabe, `fx_gain.vhd` verändert die Signalamplitude, `fx_clipping.vhd` begrenzt das Signal und die einfache Laufzeitsteuerung erfolgt durch `button_onepulse.vhd`. Weitere Effektmodule können später an derselben Stelle in die Kette eingefügt oder gegen bestehende Module ausgetauscht werden.

5.2.3 Funktionsnachweis und Bewertung

Der Funktionsnachweis erfolgte durch das Einspeisen eines externen Audiosignals am Line-In-Eingang des Pmod-I2S2 und die anschließende Wiedergabe des verarbeiteten Signals über den Audioausgang. Im Vergleich zum vorherigen Bypass war nun eine gezielte Veränderung des Klangs hörbar. Durch Erhöhen der Verstärkung wurde das Signal stärker angesteuert und dadurch auch intensiver in die Clipping-Stufe gefahren, wodurch eine deutlichere Verzerrung entstand. Bei geringerer Verstärkung fiel der Effekt entsprechend schwächer aus.

Damit konnte gezeigt werden, dass die Effektkette grundsätzlich wie vorgesehen arbeitet und die einzelnen Module korrekt zusammenwirken. Der I²S-Transceiver stellt die empfangenen Audiodaten bereit, die Verstärkungsstufe verändert deren Pegel, die Clipping-Stufe begrenzt das Signal, und die bearbeiteten Samplewerte werden anschließend wieder an den DAC

ausgegeben. Besonders wichtig an diesem Arbeitspaket ist weniger die Komplexität der einzelnen Effekte als vielmehr die erfolgreich aufgebaute Struktur. Der zuvor verifizierte Echtzeitpfad wurde um eine erste echte Hardware-Signalverarbeitung erweitert, ohne dass die grundlegende Audioübertragung verändert werden musste. Dadurch wurde gezeigt, dass sich auf dem bestehenden ADC-DAC-Pfad schrittweise weitere DSP-Funktionen ergänzen lassen.

Die Umsetzung ist bewusst einfach gehalten, erfüllt jedoch den zentralen Zweck dieses Projektstandes: Sie demonstriert, wie sich Audiomodule in einer PL-basierten Effektkette kombinieren, parametrisieren und zur Laufzeit beeinflussen lassen. Auf dieser Grundlage können zukünftig weitere Effekte ergänzt werden. Eine detailliertere Übersicht möglicher Erweiterungen wird im abschließenden Ausblick gegeben.

Aus zeitlichen Gründen werden diese Erweiterungen nicht mehr im Rahmen der HSC-Vorlesung umgesetzt. Das Projekt bietet jedoch eine gute Grundlage für eine Weiterführung über den Abgabebereich hinaus, insbesondere im Rahmen einer eigenständigen Weiterentwicklung.

Hinweis: Ein Video zur Demonstration dieses Projektstandes ist unter dem Namen „Gitarre mit und ohne Effekte Aufzeichnung“ im Google Drive hinterlegt.

6 Bekannte Probleme und Lessons Learned

Im Verlauf der Projektentwicklung traten verschiedene praktische Herausforderungen auf, die nicht nur die eigentliche Hardwarebeschreibung, sondern auch Projektstruktur, Toolchain, Takterzeugung und Inbetriebnahme betrafen. Die folgenden Punkte stellen nicht jede einzelne aufgetretene Schwierigkeit dar, sondern greifen die wichtigsten und lehrreichsten Fälle heraus. Ziel ist es, die gewonnenen Erfahrungen festzuhalten und dadurch eine spätere Weiterentwicklung des Projektes nachvollziehbarer zu machen.

6.1 Reproduzierbarkeit und Projektstruktur

Ein zentrales Problem zu Beginn des Projektes war der fehlende Fokus auf eine reproduzierbare Projektstruktur. Erste Implementierungen wurden teilweise direkt in Vivado-Projektordnern entwickelt, ohne klar zwischen handgeschriebenem Quellcode, automatisch erzeugten Dateien und temporären Build-Artefakten zu unterscheiden. Durch den Wechsel zwischen verschiedenen Rechnern entstanden dadurch schnell Probleme: Einzelne Projektstände ließen sich nicht mehr zuverlässig öffnen oder erneut aufbauen, und teilweise gingen funktionierende Zwischenstände praktisch verloren.

Die Lösung bestand darin, das Projekt konsequent in ein GitHub-Repository zu überführen und eine klarere Ordnerstruktur aufzubauen. Hardwarequellen, Constraints, Tcl-Skripte, Softwaredateien und Dokumentation wurden getrennt abgelegt. Zusätzlich wurde die Projektbeschreibung über eine `README.md` strukturiert, wodurch der aktuelle Stand und die notwendigen Schritte besser nachvollziehbar wurden.

Ein entscheidender Fortschritt war außerdem die Verwendung von Tcl-Skripten zur Erzeugung der Vivado-Projekte. Statt ein Projekt ausschließlich manuell über die grafische Oberfläche aufzubauen, konnten Projektdateien, Blockdesigns und benötigte IP-Blöcke gezielt über Skripte erzeugt werden. Dadurch ließen sich Projektstände schneller neu aufbauen, Fehler reproduzierbarer untersuchen und Versionsprobleme zwischen verschiedenen Vivado-Installationen reduzieren.

Die wichtigste Erkenntnis daraus ist, dass Reproduzierbarkeit ein wesentlicher Teil der technischen Qualität eines FPGA-Projektes ist. Ein funktionierendes Design ist deutlich wertvoller, wenn es auch später noch zuverlässig aus den Quellen und Skripten neu erzeugt werden kann.

6.2 Schrittweise Inbetriebnahme statt zu komplexer Erstaufbau

Ein weiteres Problem trat beim ersten Aufbau komplexerer Hardwaredesigns auf. Zu Beginn wurde versucht, mehrere Systembestandteile gleichzeitig zu integrieren: Processing

System, AXI-DMA, AXI-Stream-FIFO, eigener Audioblock, Takterzeugung, Wrapper-Erzeugung und Export nach Vitis. Bei auftretenden Fehlern war dadurch schwer erkennbar, ob die Ursache im Blockdesign, in der DMA-Konfiguration, im eigenen VHDL-Code, in der Taktstruktur oder in der Toolchain lag.

Als Gegenmaßnahme wurde der Aufbau in kleinere Schritte zerlegt. Zunächst wurden möglichst einfache Testdesigns erstellt, beispielsweise mit einem minimalen Audioblock und konstanten Eingangssignalen. Erst nachdem diese Basis funktionierte, wurden weitere Komponenten wie FIFO, DMA und Processing System ergänzt. Diese schrittweise Inbetriebnahme machte die Fehlersuche deutlich übersichtlicher.

Diese Erfahrung zeigte besonders deutlich, dass FPGA-Systeme nicht direkt in maximaler Komplexität aufgebaut werden sollten. Ein Stück-für-Stück-Herangehensweise ist meist schneller und zuverlässiger, weil jeder neue Systembestandteil einzeln geprüft werden kann.

6.3 Schnittstellentiming und Takterzeugung

Bei der Entwicklung der ersten Audioausgabe zeigte sich, dass eine grundsätzlich hörbare Signalausgabe noch nicht automatisch bedeutet, dass das Timing sauber umgesetzt ist. Insbesondere bei der I²S-Ausgabe ist entscheidend, wann neue Samples bereitgestellt werden und zu welchem Zeitpunkt sie vom Transceiver übernommen werden.

Eine frühe Implementierung orientierte sich noch zu stark an beobachteten Signalwechseln, beispielsweise am Zustand des Word-Select-Signals. Robuster wurde der Aufbau erst durch eine klar definierte Aktualisierung über ein eigenes `sample_tick`-Signal. Dieses markiert den Zeitpunkt, zu dem ein vollständiges Stereo-Frame verarbeitet wurde und neue Samplewerte für die nächste Ausgabe bereitgestellt werden können.

Auch die Wahl der Audiotakte erwies sich als wichtiger Punkt. Master-Clock, Bit-Clock, Word-Select-Signal und gewünschte Abtastrate hängen direkt zusammen. Unterschiedliche Taktannahmen können daher unbemerkt Einfluss auf Tonhöhe, Abtastrate oder Wiedergabeverhalten haben. Die spätere Ausrichtung der Takterzeugung auf eine konsistente Audio-Sample-Rate-Familie war daher ein wichtiger Schritt zur Stabilisierung des Systems.

Die zentrale Erkenntnis ist, dass Takterzeugung und Schnittstellentiming bei digitalen Audiosystemen keine Nebendetails sind. Gerade bei I²S muss klar definiert sein, wann Daten gültig sind, wann sie übernommen werden und in welcher Taktdomäne die jeweilige Logik arbeitet.

6.4 Clock-Domain-Crossing bei der PS/PL-Kommunikation

Bei der WAV-Wiedergabe mit DMA entstand erstmals ein System mit mehreren Takt-domänen. Die PS-/AXI-Seite arbeitet mit dem Systemtakt des Processing Systems,

während die eigentliche Audioausgabe durch den Audiotakt bestimmt wird. Anfangs war die Trennung dieser Bereiche nicht konsequent genug, wodurch schwer nachvollziehbare Fehler entstehen können, etwa blockierende AXI-Stream-Transfers oder Logik, die nicht sauber aus dem Reset-Zustand kommt.

Die Lösung bestand darin, die AXI-Stream-FIFO bewusst als Übergang zwischen diesen Taktdomänen zu verwenden. Die Eingangsseite der FIFO gehört zur AXI- beziehungsweise DMA-Domäne, während die Ausgangsseite im Audiotakt arbeitet. Dadurch wird der blockweise Datenstrom aus dem Processing System von der kontinuierlichen samplegenauen Audioausgabe entkoppelt.

Diese Erfahrung war besonders wichtig, weil sie ein typisches Problem von Hardware-Software-Codesign-Systemen zeigt: Es reicht nicht aus, Daten nur logisch von einem Block zum nächsten zu verbinden. Sobald unterschiedliche Takte beteiligt sind, muss der Übergang zwischen diesen Taktdomänen bewusst und kontrolliert ausgelegt werden.

6.5 Toolchain- und Umgebungsprobleme

Ein besonders lehrreicher Fehler trat nach der Installation von Vivado auf einem neuen Laptop auf. Ein zuvor funktionierender Projektstand stürzte bei der Synthese immer an ähnlicher Stelle ab. Zunächst sah es so aus, als läge das Problem an der Clock-Wizard-IP, an falsch erzeugten IP-Dateien oder an einer fehlerhaften Projektkonfiguration. Entsprechend wurden zunächst VHDL-Instanziierung, Reset-Signale, IP-Erzeugung und Tcl-Skripte überprüft.

Der entscheidende Wendepunkt war ein sehr kleines Testdesign, das nahezu keine eigene Logik enthielt. Auch dieses minimale Design führte zu einem Absturz. Dadurch konnte der Fehler eindeutig vom eigentlichen Audiodesign getrennt werden. Nach weiterer Eingrenzung zeigte sich, dass die Ursache nicht im Projekt selbst lag, sondern in der verwendeten Entwicklungsumgebung. Konkret war das Betriebssystem des neuen Laptops zu aktuell für die verwendete Vivado-Version. Obwohl sich die Vivado-Oberfläche normal starten ließ, war der Syntheseablauf unter dieser Systemumgebung nicht zuverlässig ausführbar. Das Problem lag damit nicht an einem fehlerhaften HDL-Entwurf, sondern an einer fehlenden beziehungsweise noch nicht ausreichend stabilen Kompatibilität zwischen Vivado und der eingesetzten Betriebssystemversion. Erst eine zukünftige Vivado-Version oder eine unterstützte ältere Systemumgebung kann dieses Problem grundsätzlich beheben.

Diese Erfahrung zeigte, dass eine sichtbare Fehlermeldung nicht immer die eigentliche Ursache beschreibt. Ein Fehler, der zunächst wie ein Problem mit einer IP, einem RTL-Modul oder einem Tcl-Skript wirkt, kann in Wirklichkeit durch eine inkompatible Toolchain verursacht werden. Ebenso sind Warnungen nicht automatisch die Ursache eines Fehlers.

Manche Warnungen sind harmlos, andere sind nur Folgefehler, und wieder andere lenken von der eigentlichen Ursache ab. Deshalb ist es wichtig, Logmeldungen kritisch zu bewerten und nicht nur der auffälligsten Fehlermeldung zu folgen.

Ein wichtiges Learning war daher, die Fehlersuche bei Bedarf bewusst anders anzugehen. Statt immer weiter am eigentlichen Projekt zu debuggen, kann ein stark reduziertes Testdesign, ein Vergleich auf einem anderen Rechner oder ein Blick auf bekannte Toolchain- und Betriebssystemprobleme deutlich schneller zur Ursache führen. Gerade bei FPGA-Projekten gehören nicht nur HDL-Code, Constraints und IP-Blöcke zum System, sondern auch Vivado-Version, Betriebssystem und Rechnerumgebung. In diesem Fall war es sinnvoller, auf eine bekannte funktionierende Entwicklungsumgebung zurückzuwechseln, statt weitere Zeit in die Fehlersuche innerhalb einer inkompatiblen Umgebung zu investieren.

6.6 Funktionale Korrektheit und wahrgenommene Audioqualität

Gerade bei Audioanwendungen zeigte sich außerdem, dass ein Signal nicht nur technisch funktionieren, sondern auch klanglich beurteilt werden muss. Ein erstes Sinussignal konnte zwar hörbar ausgegeben werden, war aber aufgrund der zunächst groben Signalerzeugung akustisch noch nicht ideal. Erst durch eine verbesserte digitale Sinuserzeugung, eine sauberere Taktung und eine klarere Trennung zwischen Signalgenerator und I²S-Transceiver wurde die Ausgabe stabiler und nachvollziehbarer.

Die wichtigste Erkenntnis aus diesem Punkt ist, dass Audioentwicklungen sowohl funktional als auch subjektiv und messtechnisch bewertet werden müssen. Ein hörbarer Ton ist ein erster Erfolg, aber erst eine saubere Struktur aus Takterzeugung, Sample-Aktualisierung, Signalqualität und reproduzierbarem Aufbau macht daraus ein verlässliches Audiosystem.

6.7 Zusammenfassung

Die aufgetretenen Probleme waren rückblickend ein wichtiger Teil des Lernprozesses. Besonders prägend waren die Bedeutung einer reproduzierbaren Projektstruktur, die Notwendigkeit einer schrittweisen Inbetriebnahme, der bewusste Umgang mit Taktdomänen sowie die Erkenntnis, dass nicht jeder Fehler im eigenen HDL-Code liegen muss. Für die Weiterentwicklung des Projektes ergibt sich daraus eine klare Konsequenz: Neue Funktionen sollten weiterhin modular aufgebaut, einzeln getestet und über reproduzierbare Skripte in das Gesamtsystem integriert werden.

7 Ausblick und kleines Fazit

Mit dem aktuellen Projektstand wurde eine funktionsfähige Grundlage für einen FPGA-basierten Audioeffektprozessor geschaffen. Die Audiokette konnte zunächst über den DAC-Pfad verifiziert, anschließend um eine softwaregestützte WAV-Wiedergabe erweitert und schließlich zu einem bidirektionalen Echtzeitpfad mit ersten PL-basierten Effektmodulen ausgebaut werden. Damit ist das grundlegende Konzept des Projektes nachgewiesen. Gleichzeitig ergeben sich aus dem erreichten Stand mehrere naheliegende Erweiterungsmöglichkeiten, die im Rahmen der aktuellen Abgabe aus zeitlichen Gründen nicht mehr umgesetzt werden konnten.

Ein zentraler nächster Schritt ist der geplante Umstieg von der bisher verwendeten Cora-Z7-Plattform auf ein Zybo Z7-20. Dieses Board wurde bereits für die Weiterführung des Projektes beschafft und bietet für ein Audio-DSP-Projekt mehrere Vorteile. Besonders relevant ist der integrierte Audio-Codec mit 3,5-mm-Audioein- und -ausgängen. Dadurch entfällt die externe Pmod-Audiolösung, und das System kann direkter als eigenständige Audioplattform genutzt werden. Zusätzlich bietet das Zybo Z7-20 mehr verfügbare Peripherie, mehr Ressourcen für die programmierbare Logik sowie eine HDMI-Schnittstelle, die für eine spätere grafische Darstellung von Audiosignalen genutzt werden kann.

Ein weiterer geplanter Entwicklungsschritt ist die Erweiterung der Signalverarbeitung um frequenzbasierte Verfahren. Insbesondere soll eine Fourier-Transformation beziehungsweise FFT eingesetzt werden, um das Audiosignal im Frequenzbereich analysieren und bearbeiten zu können. Darauf aufbauend könnten komplexere Filter und ein deutlich flexiblerer Equalizer realisiert werden. Langfristig wäre es interessant, eine Art hochwertig parametrierbaren Equalizer aufzubauen, der sich stärker an typischen Anforderungen aus dem Audio- und Studiobereich orientiert.

Neben der reinen Signalverarbeitung soll auch die Bedienbarkeit des Effektprozessors verbessert werden. Während die bisherigen Projektstände nur eine einfache Steuerung über wenige Taster verwenden, bietet eine stärkere Einbindung des Processing Systems deutlich mehr Möglichkeiten. Das PS könnte zukünftig zur Parametrierung der Effekte, zur Auswahl verschiedener Effektketten oder zur Verwaltung von Presets genutzt werden. Zusätzlich könnten die auf dem Zybo vorhandenen Taster und Schalter eingesetzt werden, um während des Betriebs zwischen Effekten umzuschalten oder Parameter direkt zu verändern.

Besonders interessant ist außerdem die geplante Nutzung der HDMI-Schnittstelle zur Visualisierung der Audiosignale. Dadurch könnte das System um eine Art einfaches Audio-Oszilloskop oder Spektrumanalyse-Werkzeug erweitert werden. Denkbar wäre beispielsweise, das Eingangssignal und das bearbeitete Ausgangssignal parallel auf einem

Monitor darzustellen. Ebenso könnte mithilfe der FFT das Frequenzspektrum angezeigt werden, um sichtbar zu machen, welche Wirkung einzelne Effekte oder Filter tatsächlich auf das Signal haben. Gerade für die Entwicklung und Bewertung von DSP-Effekten wäre eine solche visuelle Rückmeldung sehr hilfreich.

Langfristig kann sich das Projekt damit von einem kleinen Vorlesungsprojekt zu einem eigenständigen Audio-DSP-Experimentiertool weiterentwickeln. Die ursprüngliche Idee eines kleinen Effektgerätes entstand aus dem Interesse an Gitarrensound und Mikrocontrollern. Im Laufe des Projektes entwickelte sich daraus jedoch zunehmend eine Begeisterung für die technischen Möglichkeiten, die mit einem FPGA bereits aus der Schachtel heraus realisierbar sind.

Die Grundsteine sind mit dem aktuellen Projektstand gelegt. Die nächsten Schritte bestehen darin, die Plattform um weitere Effekte zu erweitern, komfortabler steuerbar zu machen und sowohl klanglich als auch technisch weiter zu verbessern. Ein besonders wichtiger Schritt ist dabei die Umsetzung eines geeigneten Instrumenteneingangs für ein Standard-Gitarrenkabel. Um das rohe Gitarrensignal sinnvoll nutzbar zu machen, wird eine zusätzliche analoge Eingangsstufe benötigt, beispielsweise zur Impedanzanpassung und Pegelaufbereitung. Gerade dieser Punkt bietet eine sehr gute Möglichkeit, das Projekt zukünftig auch um schaltungstechnische Erfahrung zu erweitern.

Abschließend lässt sich sagen, dass hinter den drei Worten „Was wäre wenn?“, gefolgt von einer einfachen Idee, oft ein unberechenbares Abenteuer steckt. Aus einem zunächst naiven Gedanken entstehen Stunden voller Frust, Fehlersuche und Kopfschmerzen. Gelingt es jedoch, einen entscheidenden Schritt umzusetzen, entsteht daraus ein glücklicher Moment, begleitet von einem hoffnungsvollem Gefühl für das Potenzial eigener Ideen. Mit diesen drei Worten sollte man also vorsichtig sein – sie können der Anfang eines deutlich größeren Projektes sein, als man zunächst erwartet.

8 Literatur

Digilent Inc.: Pmod I2S2 Reference Manual, <https://digilent.com>, aufgerufen am 9.3.2026.

Abbildungsverzeichnis

1	DSP - Signaleingangskette	1
2	Schematische Darstellung der Puls-Code-Modulation (PCM)	3
3	Schematische Darstellung einer bipolaren Pulsweitenmodulation (PWM)	4
4	Schematische Darstellung der I ² S-Verbindung zwischen FPGA-I ² S-Sender und externem DAC auf dem Pmod-I2S2.	7
5	Zeitdiagramm der I ² S-Datenübertragung eines Stereo-Frames ³	7
6	Verwendete Hardwareplattform: (1) Cora-Z7-Entwicklungsboard mit Zynq-SoC (2) Pmod-I2S2 als externe Audio-Peripherie	9
7	Struktur des Repositorys	12
8	Vivado Projekt erstellen	13
9	Erforderliche Dateistruktur im <code>src</code> -Verzeichnis der Vitis-Applikation.	16
10	Frequenzmessung des erzeugten Testsignals	19
11	Vereinfachtes Blockdiagramm der WAV-basierten Audiowiedergabe mit Aufgabentrennung zwischen PS und PL.	24
12	Schematische Wirkung der Verstärkungsstufe auf ein sinusförmiges Eingangssignal.	31
13	Schematische Wirkung einer Hard-Clipping-Stufe auf ein sinusförmiges Eingangssignal.	31
14	Schematische Wirkung einer Soft-Clipping-Stufe auf ein sinusförmiges Eingangssignal.	32